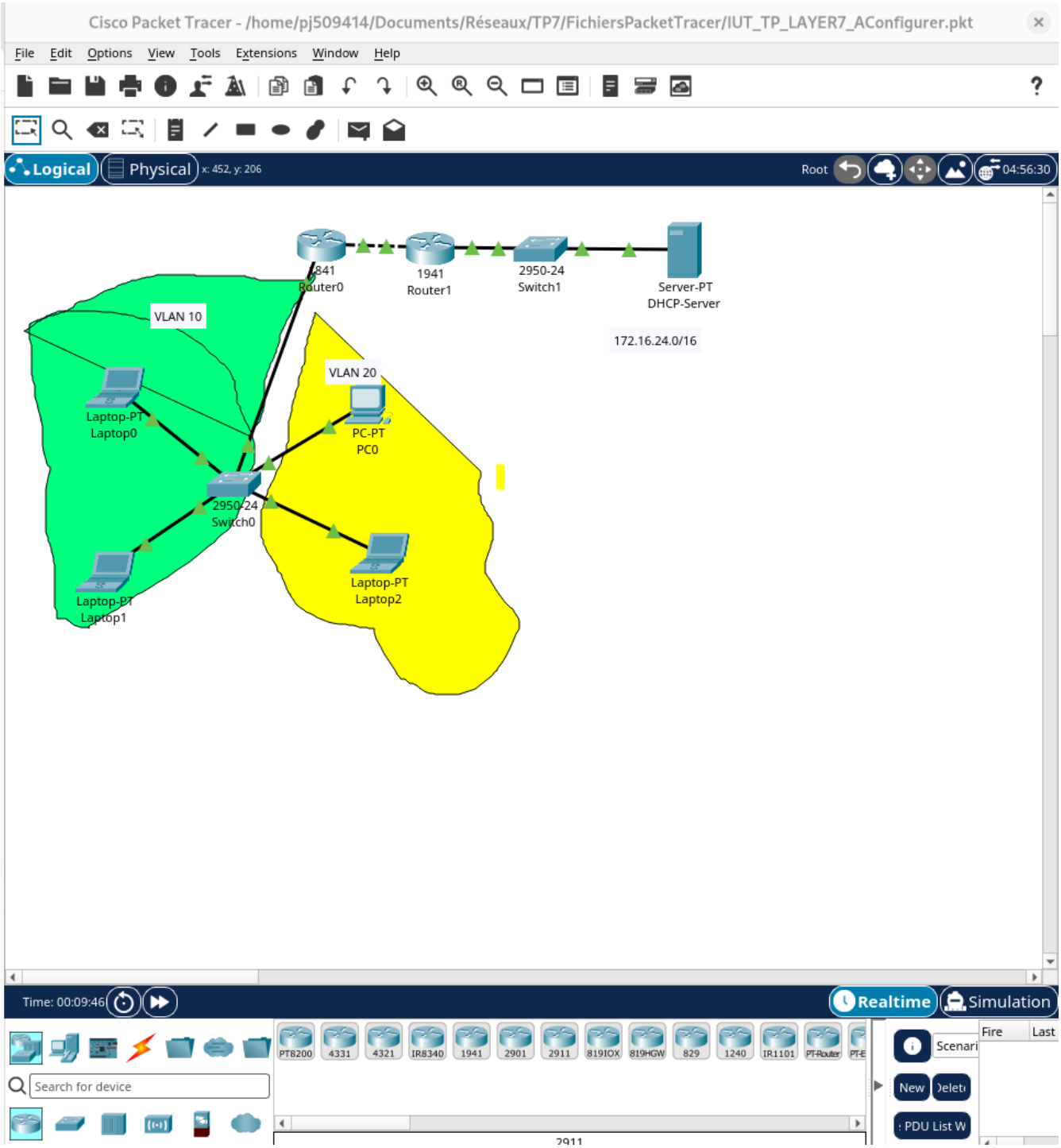
TP7 — Applications

Johan Polsinelli — BUT2 Informatique

Partie I — DHCP, HTTP et DNS

0. Topologie et plan d'adressage



Topologie de départ

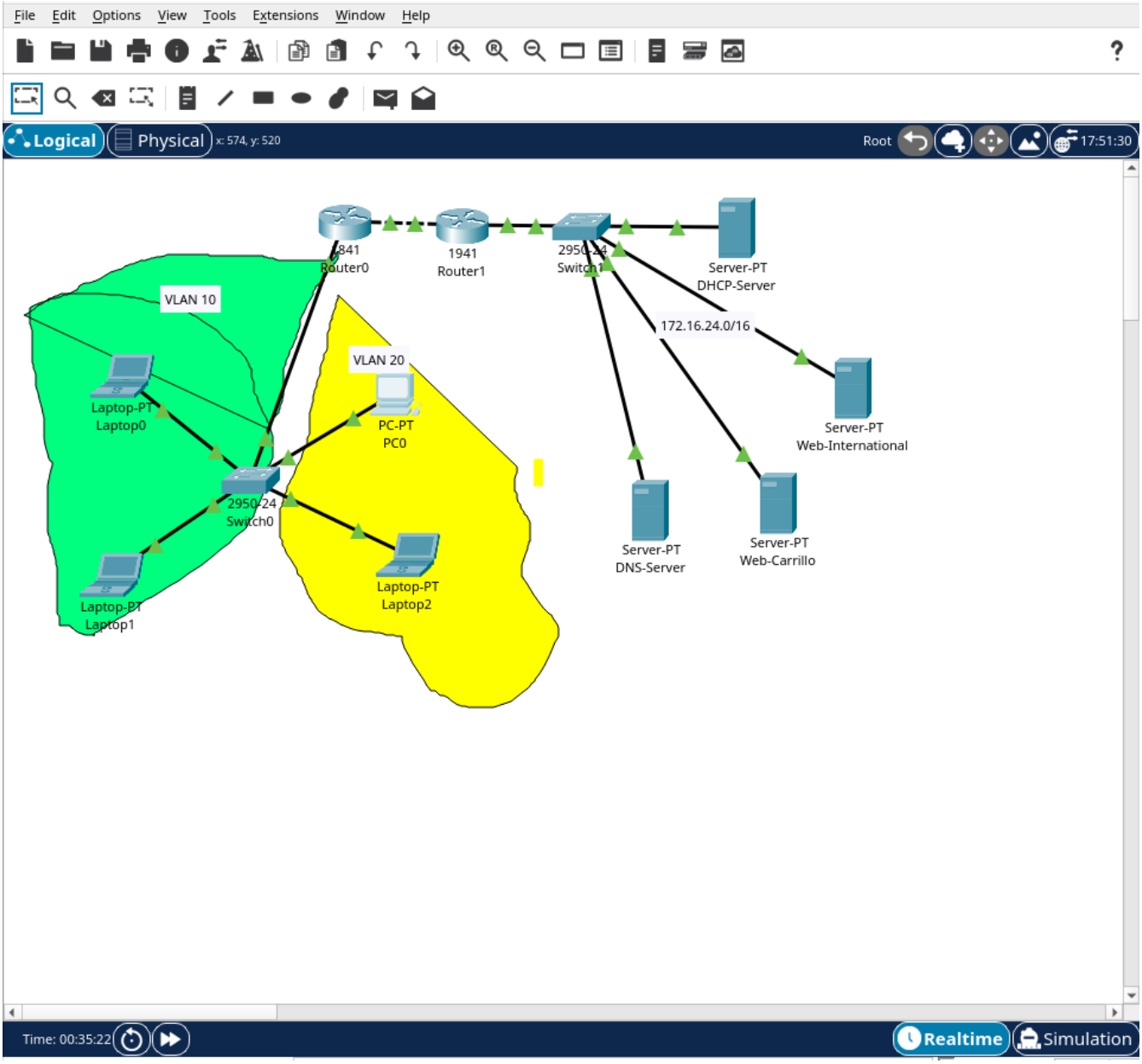
La maquette de départ contient 4 postes, 2 switchs, 2 routeurs et 1 serveur. Deux VLAN sont **déjà** configurés sur `Switch0`, et les sous-interfaces du routeur sont créées mais **sans adresse IP**. Relevé de la configuration du switch : `Fa0/1` et `Fa0/2` en VLAN 10, `Fa0/3` et `Fa0/4` en VLAN 20, `Fa0/5` en **trunk** vers `Router0`.

Plan d'adressage utilisé (celui du sujet et de la figure) :

Réseau	Adresse	Passerelle
VLAN 10 (vert)	`192.168.10.0/24`	`Router0` Fa0/0.10 = `192.168.10.1`
VLAN 20 (jaune)	`192.168.20.0/24`	`Router0` Fa0/0.20 = `192.168.20.1`
LAN des serveurs	`172.16.24.0/16`	`Router1` Gig0/1 = `172.16.24.1`
Liaison entre routeurs	`10.6.8.0/30`	`10.6.8.1` ↔ `10.6.8.2`

Équipement	Adresse	Rôle
`DHCP-Server` (ex-`Server0`)	`172.16.24.12`	serveur DHCP (les 3 pools)
`DNS-Server` (ajouté)	`172.16.24.11`	serveur DNS (2 enregistrements `A`)
`Web-Carrillo` (ajouté)	`172.16.24.20`	serveur web `www.carrillo.edu`
`Web-International` (ajouté)	`172.16.24.30`	serveur web `www.international.com`

On renomme le serveur existant en `DHCP-Server` et on ajoute **3** `Server-PT` câblés en câble droit sur les ports libres de `Switch1` (`Fa0/3`, `Fa0/4`, `Fa0/5`).



Topologie finale

1. Configurer le service DHCP pour le réseau 172.16.24.0/16

Le serveur DHCP reçoit une **adresse fixe** `172.16.24.12/16` avec la passerelle `172.16.24.1` (c'est un service : il ne peut pas dépendre du DHCP pour son adresse).

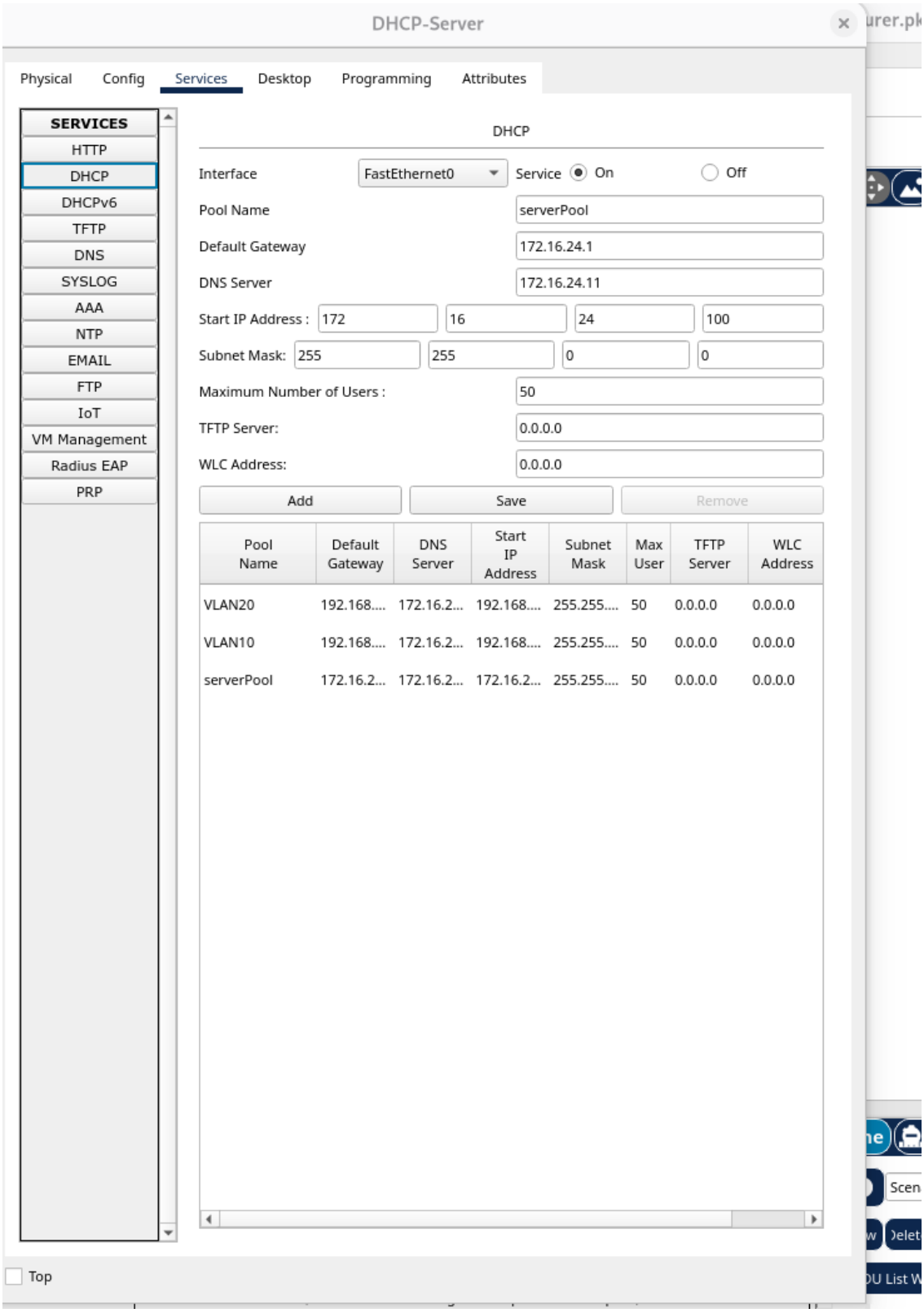
Le service est activé dans l'onglet **Services → DHCP** (Service : On), puis on crée le pool du réseau des serveurs avec `Add` **puis** `Save`. Pour chaque pool on renseigne aussi **Maximum Number of Users =** `50` ; `TFTP Server` et `WLC Address` restent à `0.0.0.0`, on ne s'en sert pas.

Pool	Default Gateway	DNS Server	Start IP	Subnet Mask
`serverPool`	172.16.24.1	172.16.24.11	172.16.24.100	255.255.0.0

2. Configurer le serveur pour les deux VLAN

Comme le serveur DHCP n'est pas dans les VLAN, il faut **un pool par sous-réseau** : chaque pool indique la passerelle de son VLAN (c'est elle qui sera donnée aux postes).

Pool	Default Gateway	DNS Server	Start IP	Subnet Mask
VLAN10	192.168.10.1	172.16.24.11	192.168.10.100	255.255.255.0
VLAN20	192.168.20.1	172.16.24.11	192.168.20.100	255.255.255.0



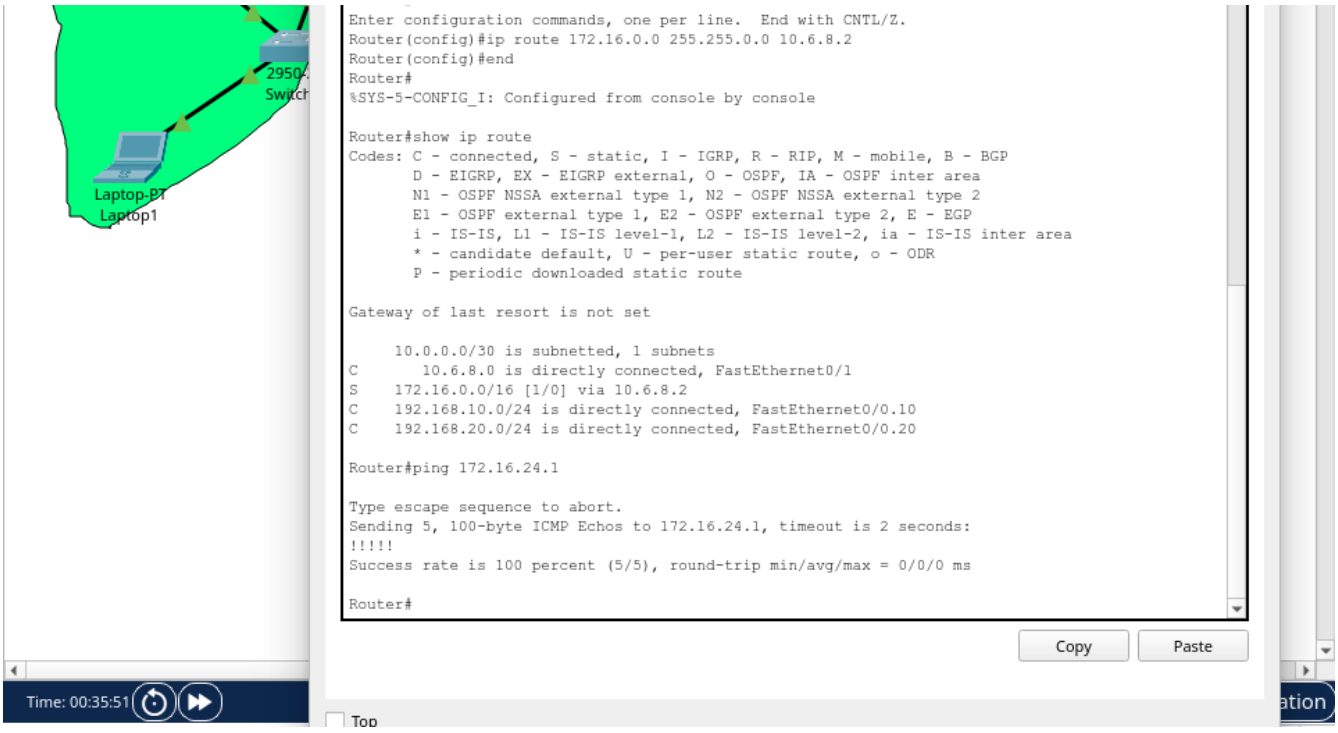
DHCP-Server : le service activé et les 3 pools

3. Indiquer au routeur la « UDP Helper Address » (`ip helper-address`)

Un poste qui démarre n'a pas encore d'adresse IP : il envoie sa requête DHCP en **broadcast** (`255.255.255.255`). Or un routeur **ne transmet jamais un broadcast** d'un réseau à un autre. Comme le serveur est sur le réseau `172.16.24.0/16`, il faut que `Router0` transforme ce broadcast en **unicast** vers `172.16.24.12` : c'est le rôle de `ip helper-address`, à mettre sur **chaque sous-interface de VLAN**.

Configuration complète de `Router0` (onglet CLI) :

```
enable
configure terminal
interface FastEthernet0/0
 no shutdown
 exit
interface FastEthernet0/0.10
 encapsulation dot1Q 10
 ip address 192.168.10.1 255.255.255.0
 ip helper-address 172.16.24.12
 exit
interface FastEthernet0/0.20
 encapsulation dot1Q 20
 ip address 192.168.20.1 255.255.255.0
 ip helper-address 172.16.24.12
 exit
ip routing
ip route 172.16.0.0 255.255.0.0 10.6.8.2
interface FastEthernet0/1
 ip address 10.6.8.1 255.255.255.252
 no shutdown
 end
show ip route
```



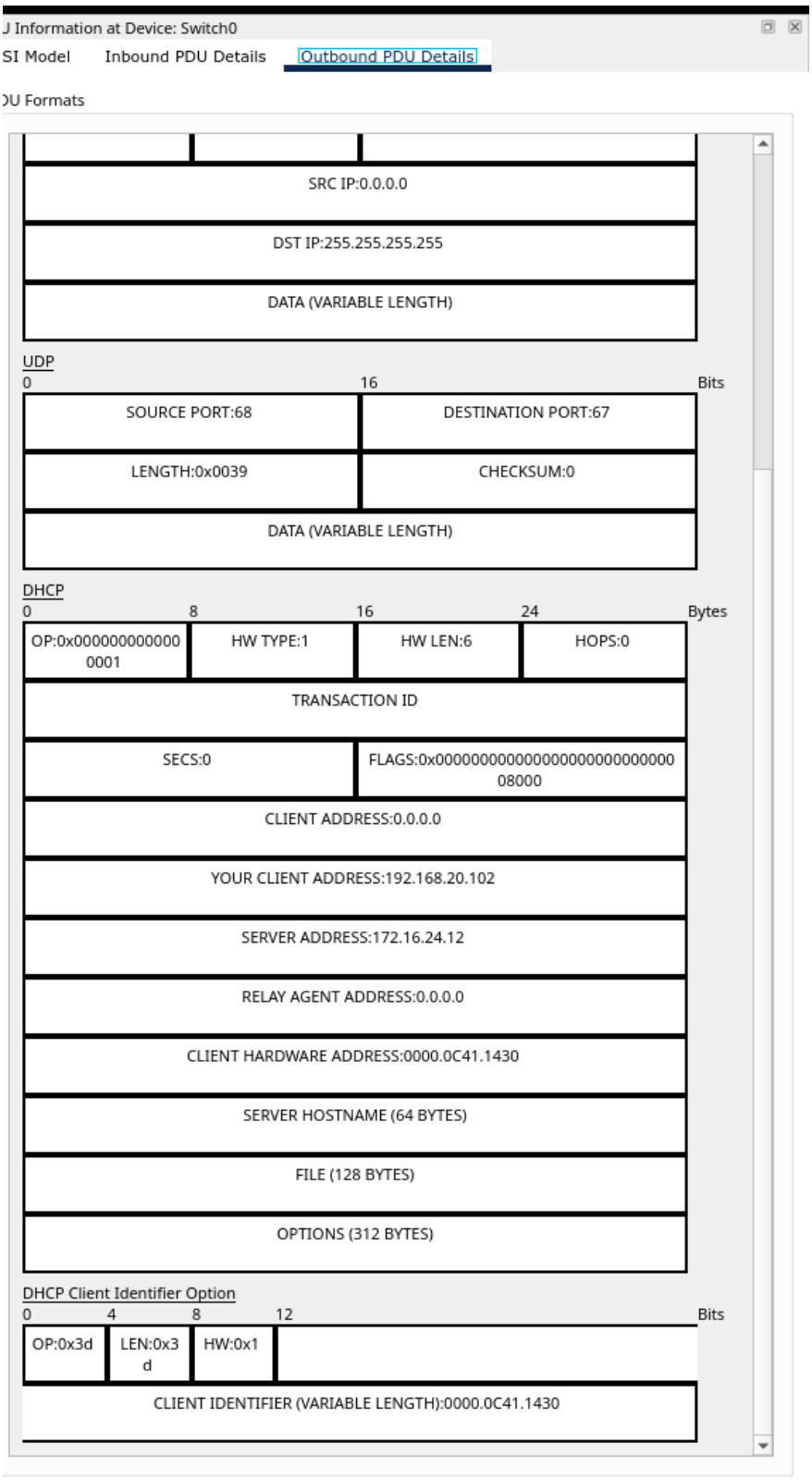
Router0 : route statique et vérification

La sortie `show ip route` de `Router0` :

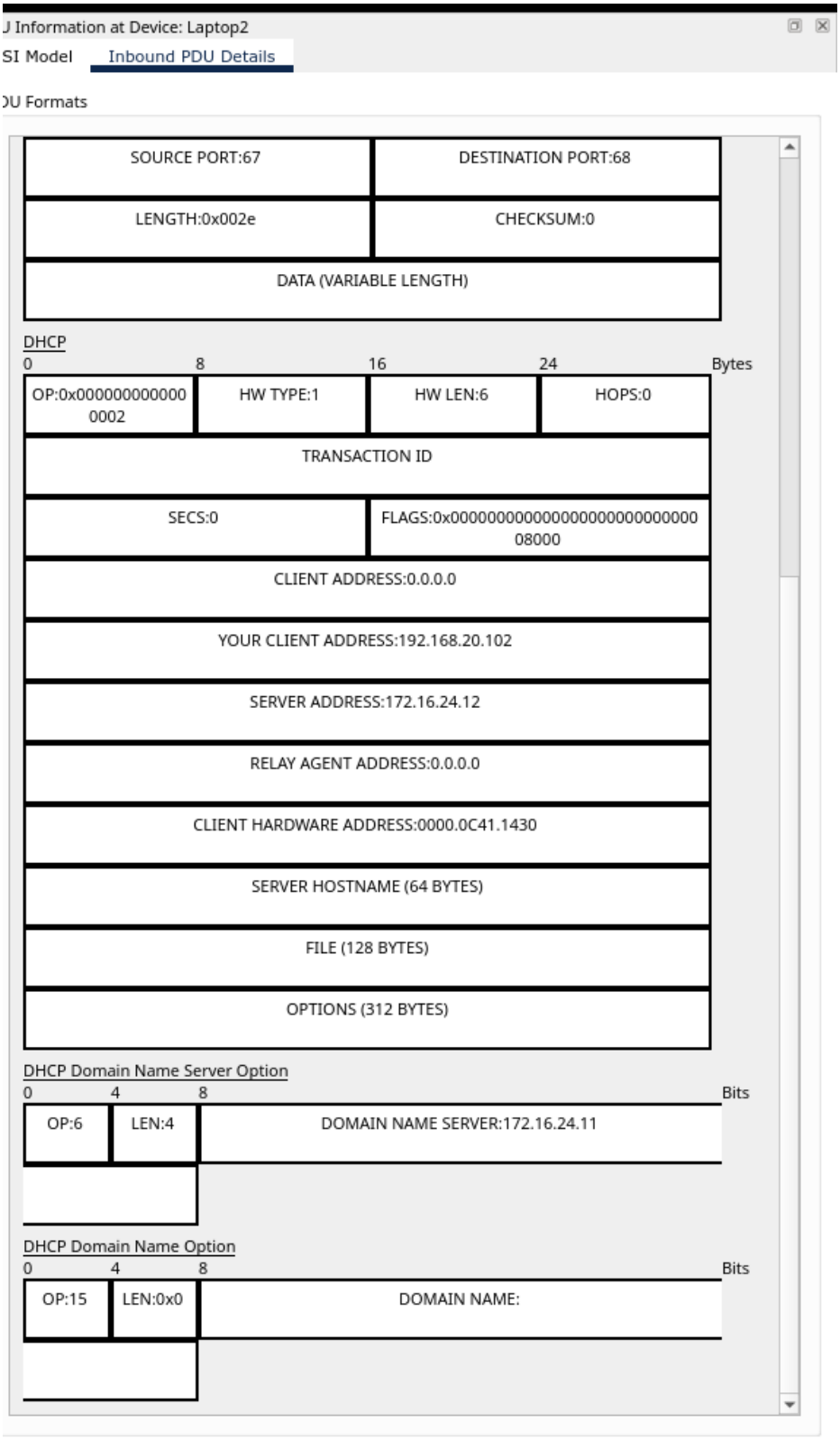
```
C    10.6.8.0/30 is directly connected, FastEthernet0/1
S    172.16.0.0/16 [1/0] via 10.6.8.2
C    192.168.10.0/24 is directly connected, FastEthernet0/0.10
C    192.168.20.0/24 is directly connected, FastEthernet0/0.20
```

et `Router0#ping 172.16.24.1` répond **5/5 (100 %)**, `round-trip 0/0/0 ms` : le LAN des serveurs est joignable à travers `Router1`.

Le relais fonctionne : le poste, qui n'a pas encore d'adresse, envoie sa demande **en broadcast** (`0.0.0.0` → `255.255.255.255`, UDP 68 → 67), le routeur la transforme en unicast vers `172.16.24.12` — c'est le `ip helper-address` — et la réponse revient avec l'adresse attribuée et l'option DNS.



Le poste envoie sa demande DHCP en broadcast



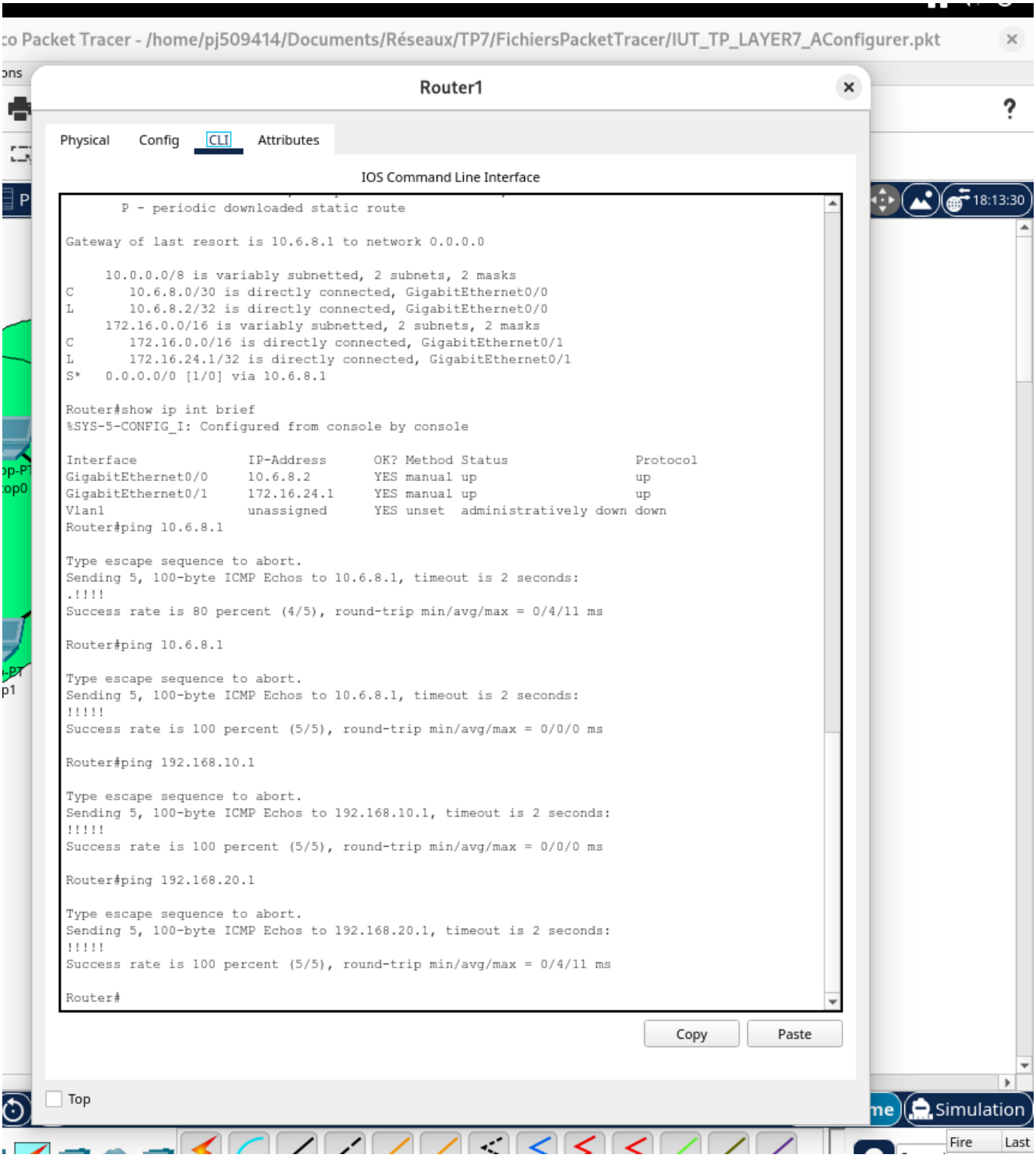
La réponse du serveur DHCP : adresse attribuée et DNS

Sur la réponse on lit `YOUR CLIENT ADDRESS: 192.168.20.102` (l'adresse donnée au poste), `SERVER ADDRESS: 172.16.24.12` et l'option 6 `DHCP Domain Name Server Option: 172.16.24.11` : c'est bien le DHCP qui transmet l'adresse du serveur DNS aux postes.

`Router1` est adressé de la même façon (WAN sur `Gig0/0`, LAN des serveurs sur `Gig0/1`) avec une route par défaut :

```
interface GigabitEthernet0/0
 ip address 10.6.8.2 255.255.255.252
```

```
 no shutdown
interface GigabitEthernet0/1
 ip address 172.16.24.1 255.255.0.0
 no shutdown
ip route 0.0.0.0 0.0.0.0 10.6.8.1
```



Router1 : interfaces, route par défaut et pings

Relevés sur `Router1` :

Commande	Résultat mesuré
`show ip route`	`S* 0.0.0.0/0 [1/0] via 10.6.8.1, C 10.6.8.0/30 directly connected, GigabitEthernet0/0, C 172.16.0.0/16 directly connected, GigabitEthernet0/1`

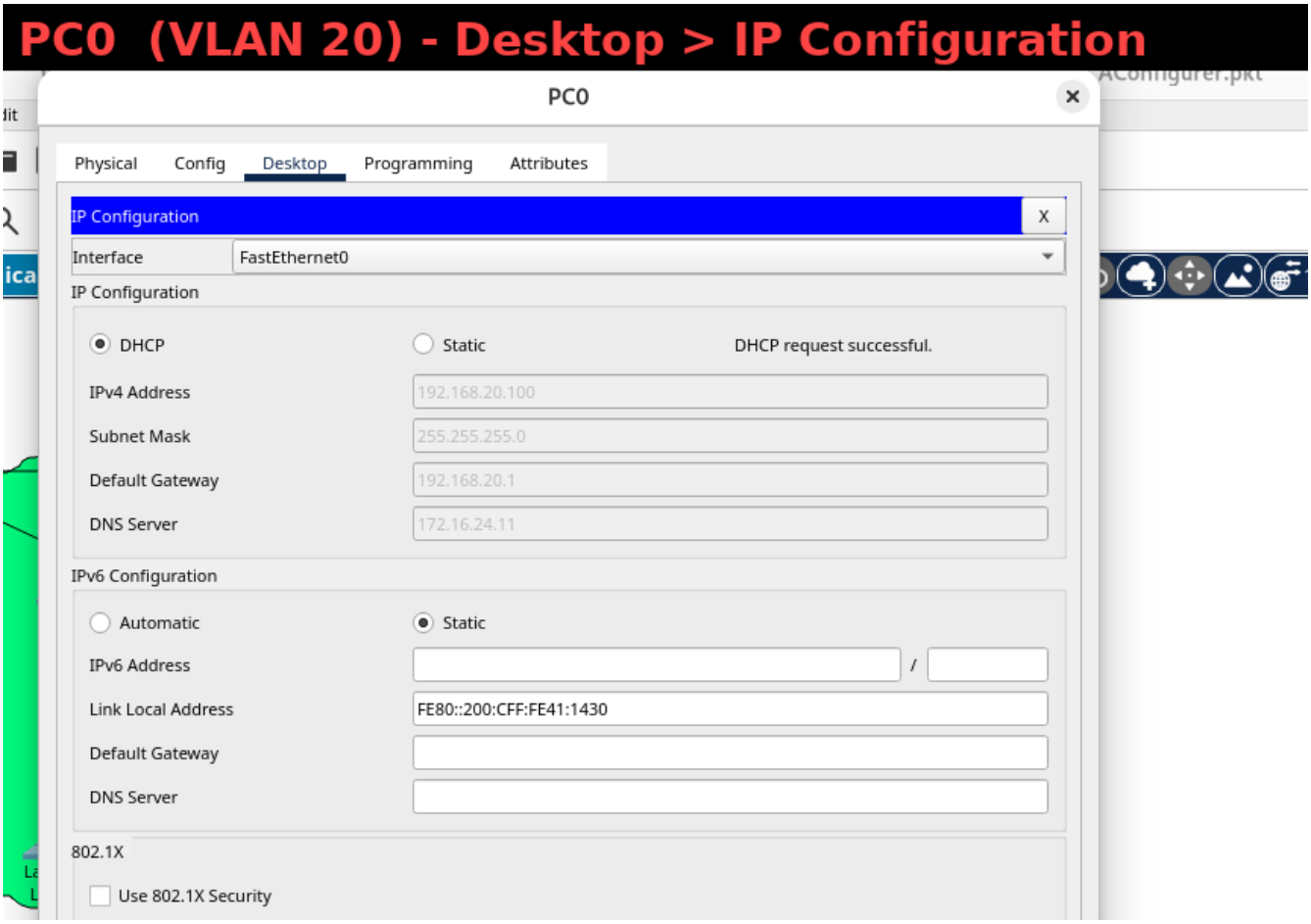
Commande	Résultat mesuré
`show ip int brief`	`Gig0/0 10.6.8.2 up/up`, `Gig0/1 172.16.24.1 up/up`, `Vlan1 administratively down`
`ping 10.6.8.1`	4/5 (80 %) au 1er essai (ARP), puis **5/5 (100 %)**
`ping 192.168.10.1`	**5/5 (100 %)** — le VLAN 10 répond
`ping 192.168.20.1`	**5/5 (100 %)** — le VLAN 20 répond

---

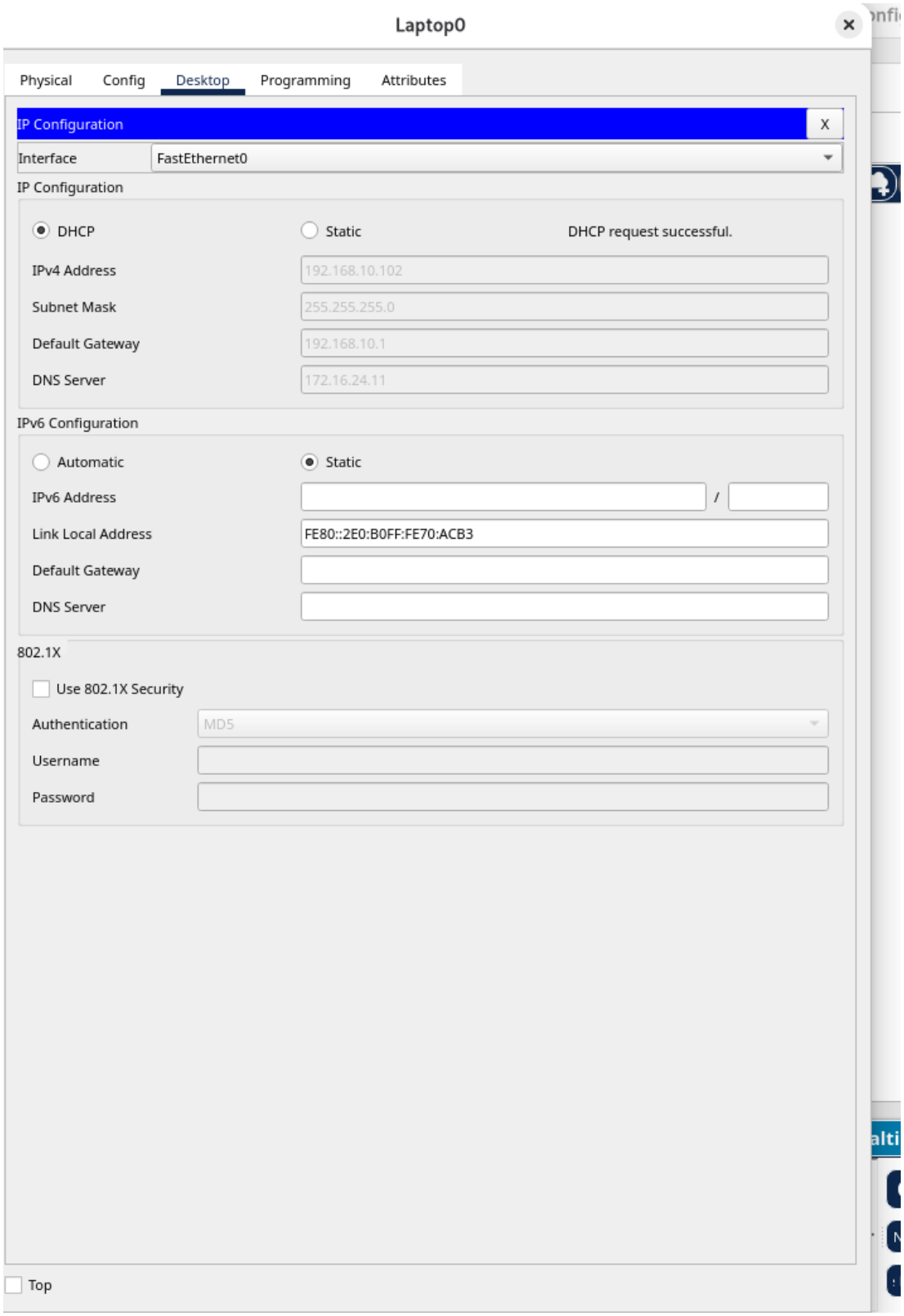
4. Faire les tests avec différents ping

On met les 4 postes en **DHCP** (`Desktop → IP Configuration` → DHCP) et on relève les adresses réellement obtenues :

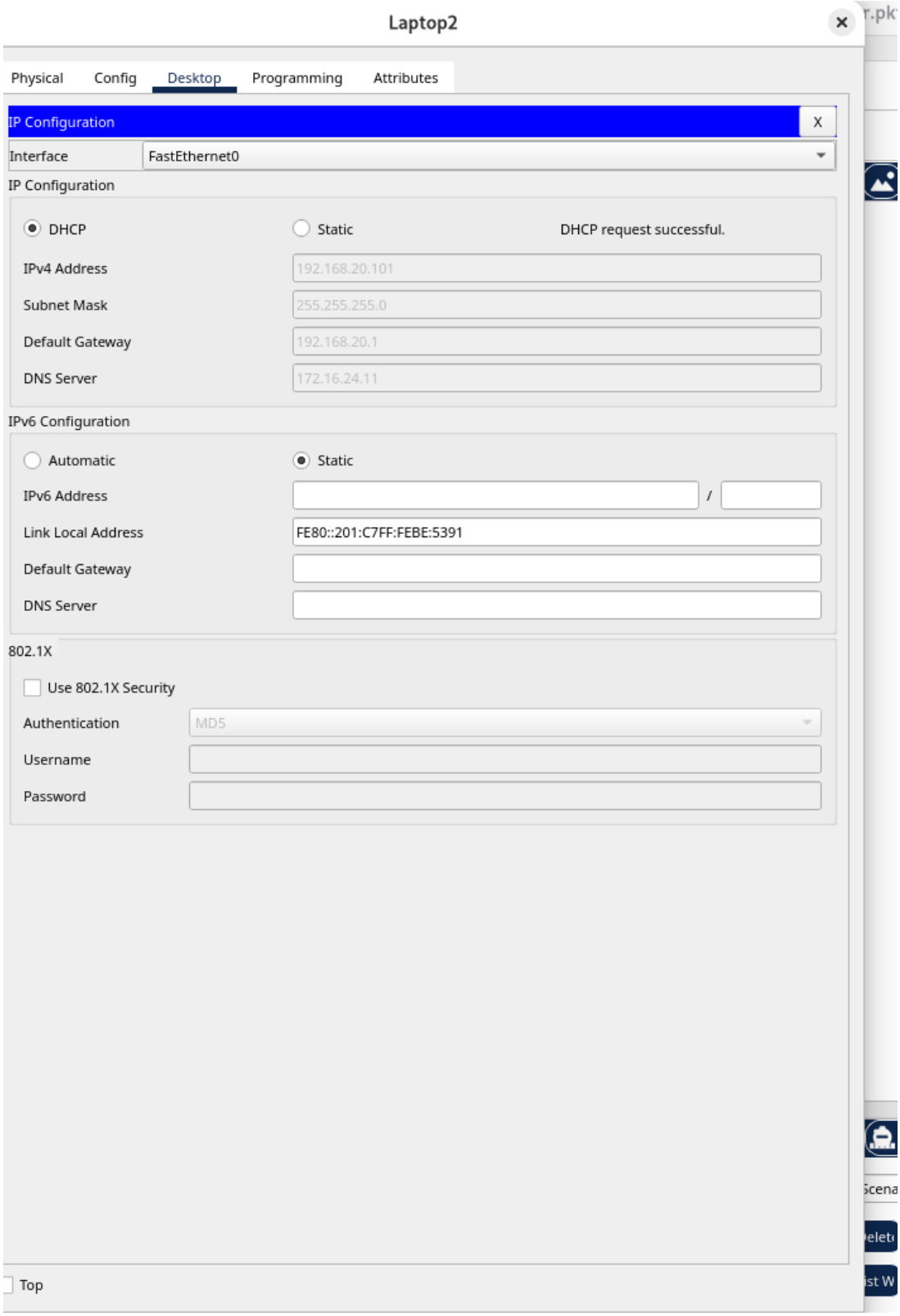
Poste	Adresse obtenue	Passerelle	DNS
PC0	192.168.20.100	192.168.20.1	172.16.24.11
Laptop2	192.168.20.101	192.168.20.1	172.16.24.11
Laptop0	192.168.10.102	192.168.10.1	172.16.24.11
Laptop1	192.168.10.101	192.168.10.1	172.16.24.11



PC0 : adresse obtenue en DHCP

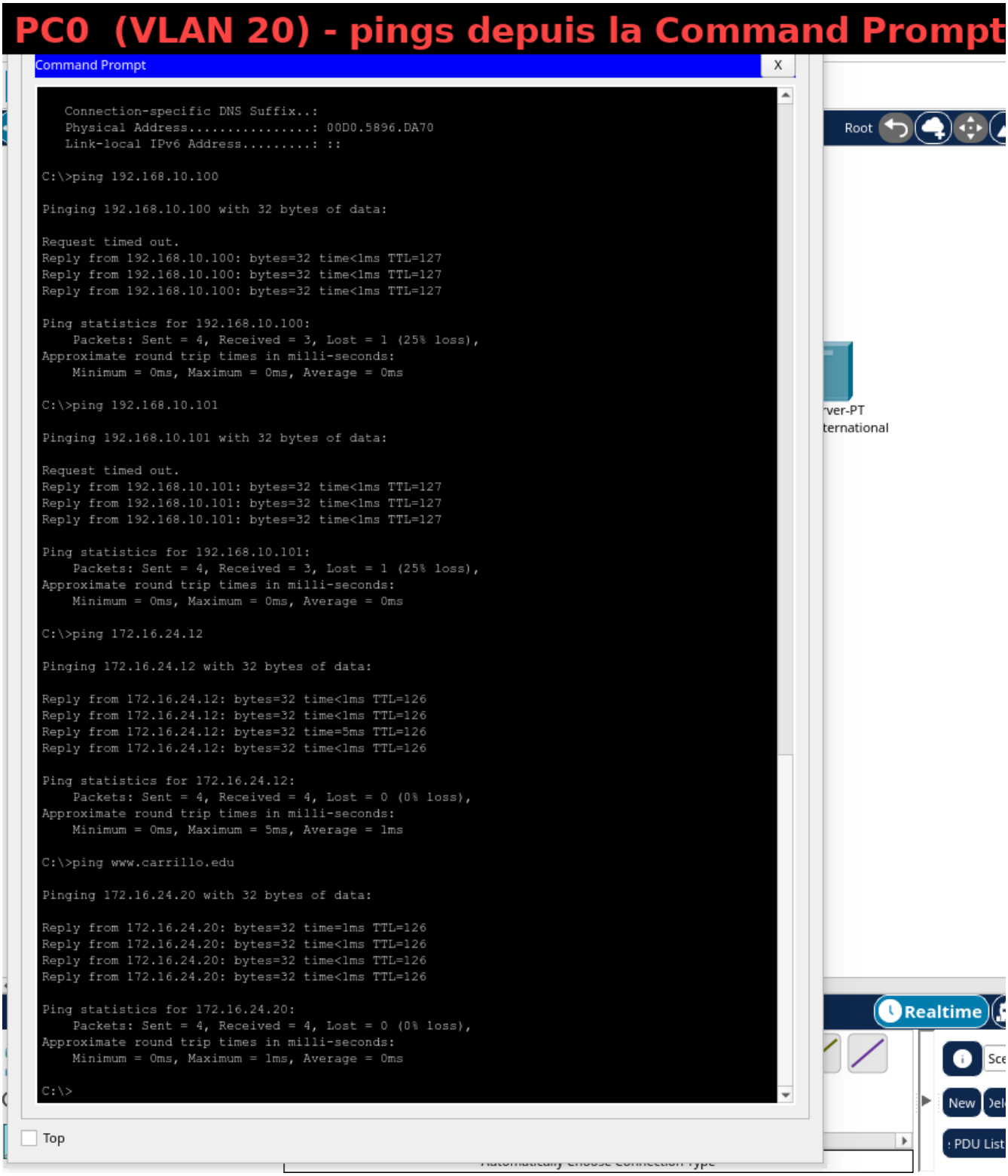


Laptop0 : adresse obtenue en DHCP



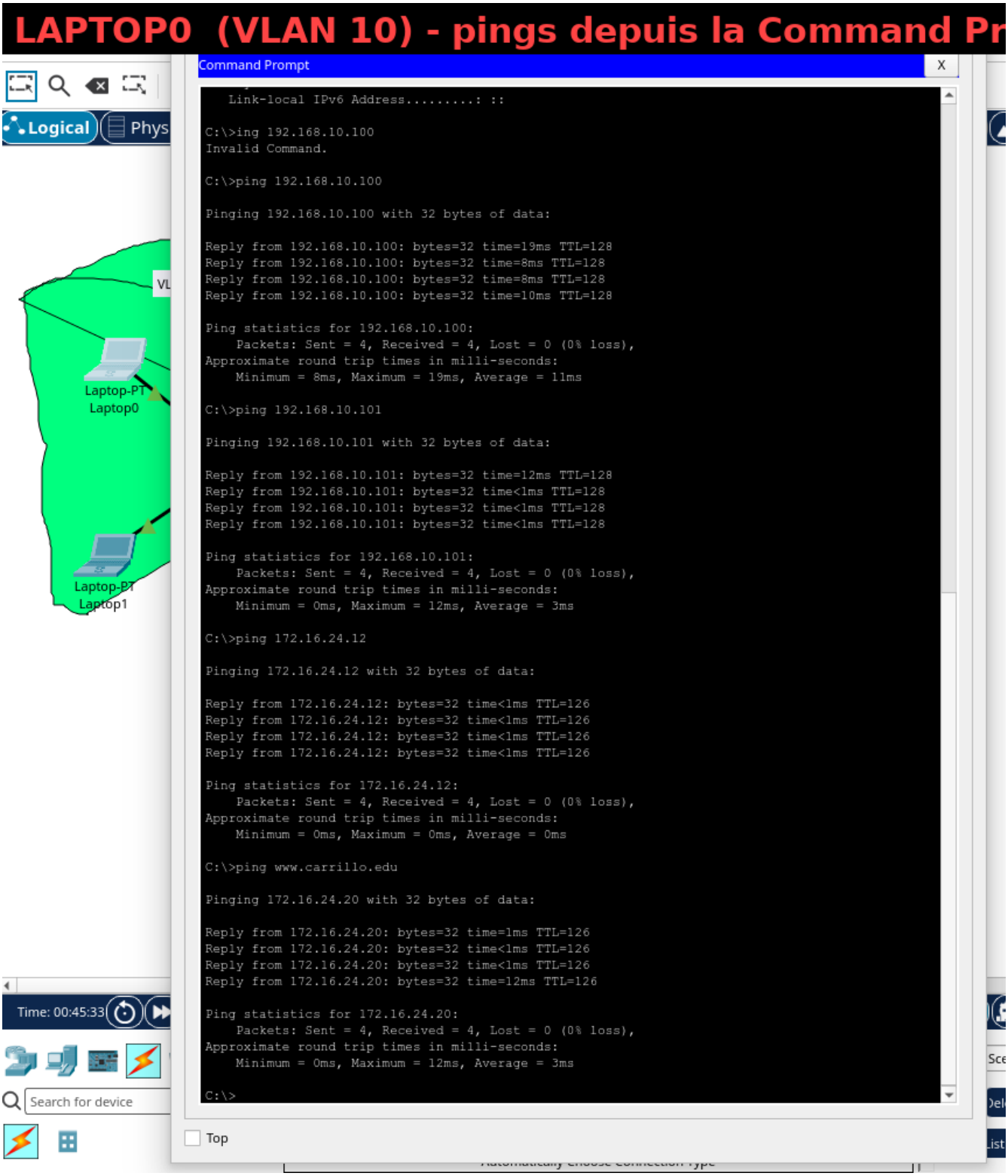
Laptop2 : adresse obtenue en DHCP

a) Depuis PC0 (VLAN 20) vers le VLAN 10, vers le serveur DHCP et par nom :



Pings depuis PC0

b) Depuis Laptop0 (VLAN 10) :



Pings depuis Laptop0

Résultats obtenus, relevés dans les captures.

Depuis `PC0` (`192.168.20.100`, VLAN 20) :

Test	Résultat mesuré
`ping 192.168.10.100`	3/4 — 1 perdu (25 %), réponses **TTL 127**
`ping 192.168.10.101`	3/4 — 1 perdu (25 %), réponses **TTL 127**
`ping 172.16.24.12`	**4/4 (0 %)**, **TTL 126**, min 0 / max 5 / moy 1 ms

Test	Résultat mesuré
`ping www.carrillo.edu`	résolu en `172.16.24.20`, **4/4 (0 %)**, **TTL 126**

Depuis `Laptop0` (`192.168.10.102`, VLAN 10) :

Test	Résultat mesuré
`ping 192.168.10.100`	**4/4 (0 %)** — c'est la passerelle du VLAN 10 (`Router0`), **TTL 128**, 8 / 19 / 11 ms
`ping 192.168.10.101`	**4/4 (0 %)**, **TTL 128** — `Laptop1`, même VLAN, aucun routeur traversé
`ping 172.16.24.12`	**4/4 (0 %)**, **TTL 126**
`ping www.carrillo.edu`	résolu en `172.16.24.20`, **4/4 (0 %)**, **TTL 126**

Le **TTL** mesure la distance : `128` = aucun routeur traversé (les deux postes sont dans le VLAN 10), `127` = un routeur (`Router0` entre les deux VLAN), `126` = deux routeurs (`Router0` + `Router1` jusqu'au LAN des serveurs). Le premier écho est parfois perdu (« Request timed out ») : c'est la résolution ARP avant la première réponse ; les suivants passent à 100 %.

---

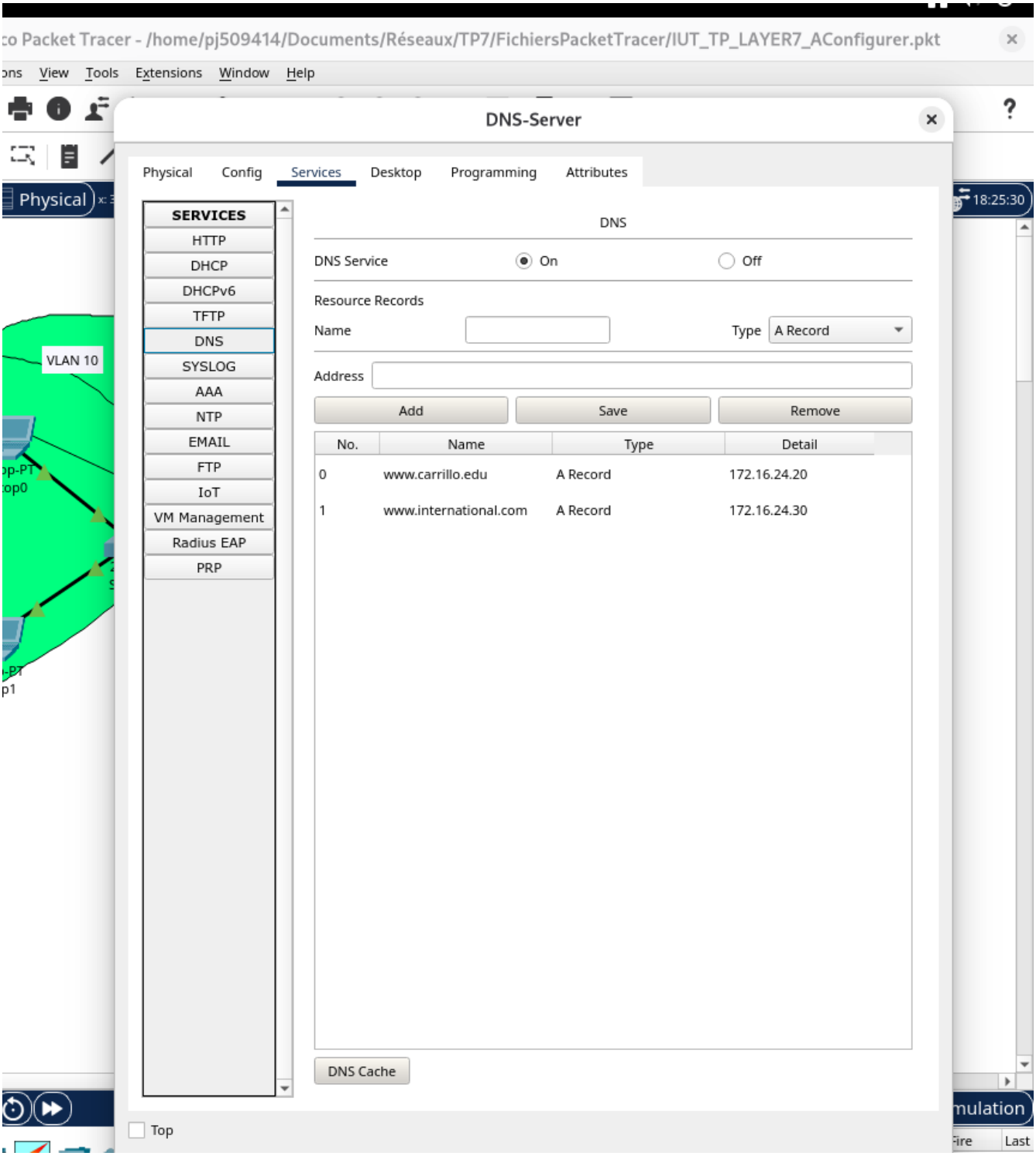
5. Ajouter et activer les serveurs web et le serveur DNS

Chaque serveur est adressé en **statique** avec la passerelle `172.16.24.1`, puis le service est activé dans l'onglet **Services** de l'équipement.

Le serveur DNS

Services → DNS → DNS Service : On, puis **un** `A Record` **par serveur web** :

Name	Type	Address
`www.carrillo.edu`	A Record	`172.16.24.20`
`www.international.com`	A Record	`172.16.24.30`

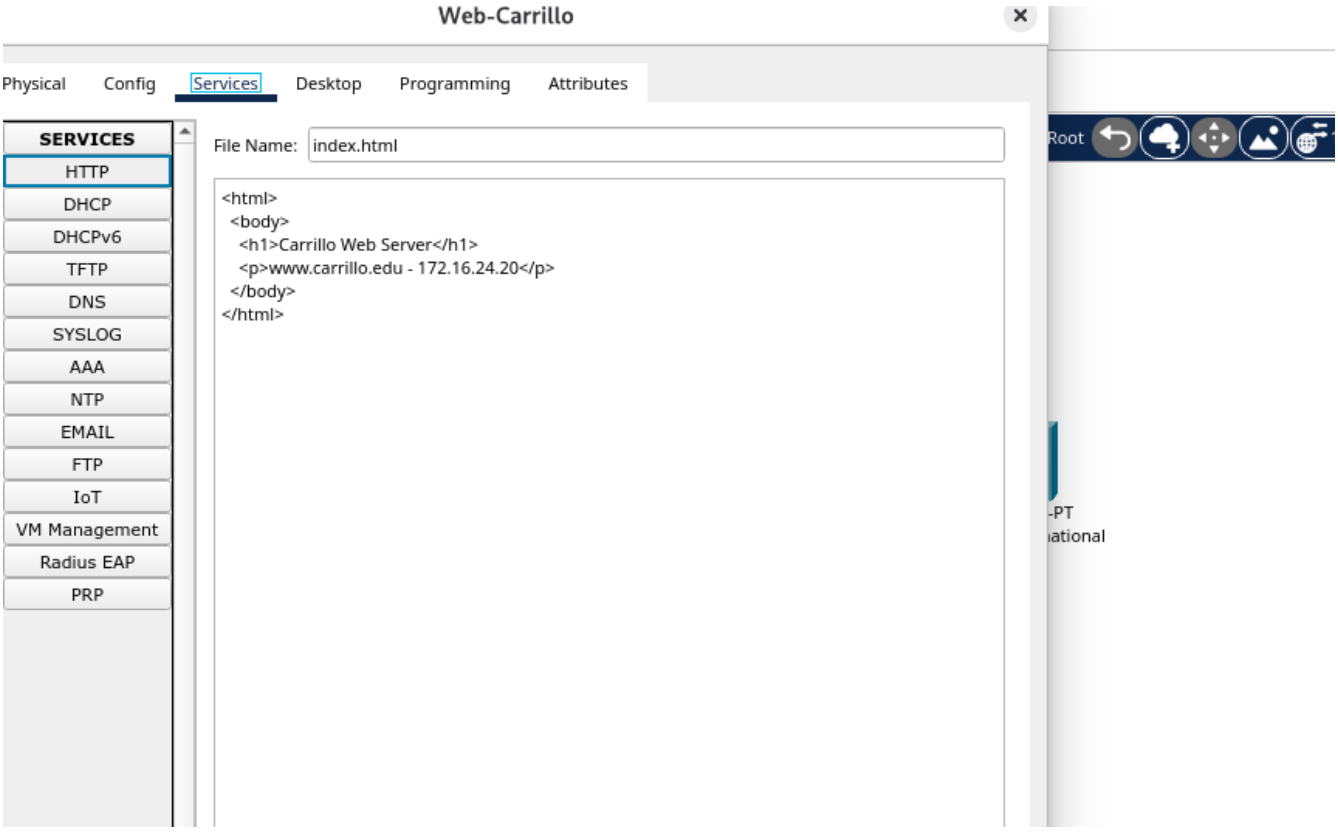


Serveur DNS : les deux enregistrements

Les serveurs web

Services → HTTP → HTTP : On, et on remplace le contenu de `index.html` par une page qui identifie le serveur (pour bien voir lequel répond) :

```
<html>
  <body>
    <h1>Carrillo Web Server</h1>
    <p>www.carrillo.edu - 172.16.24.20</p>
  </body>
</html>
```



Serveur web : HTTP activé et page modifiée

6. Que devez-vous changer au niveau du serveur DHCP ?

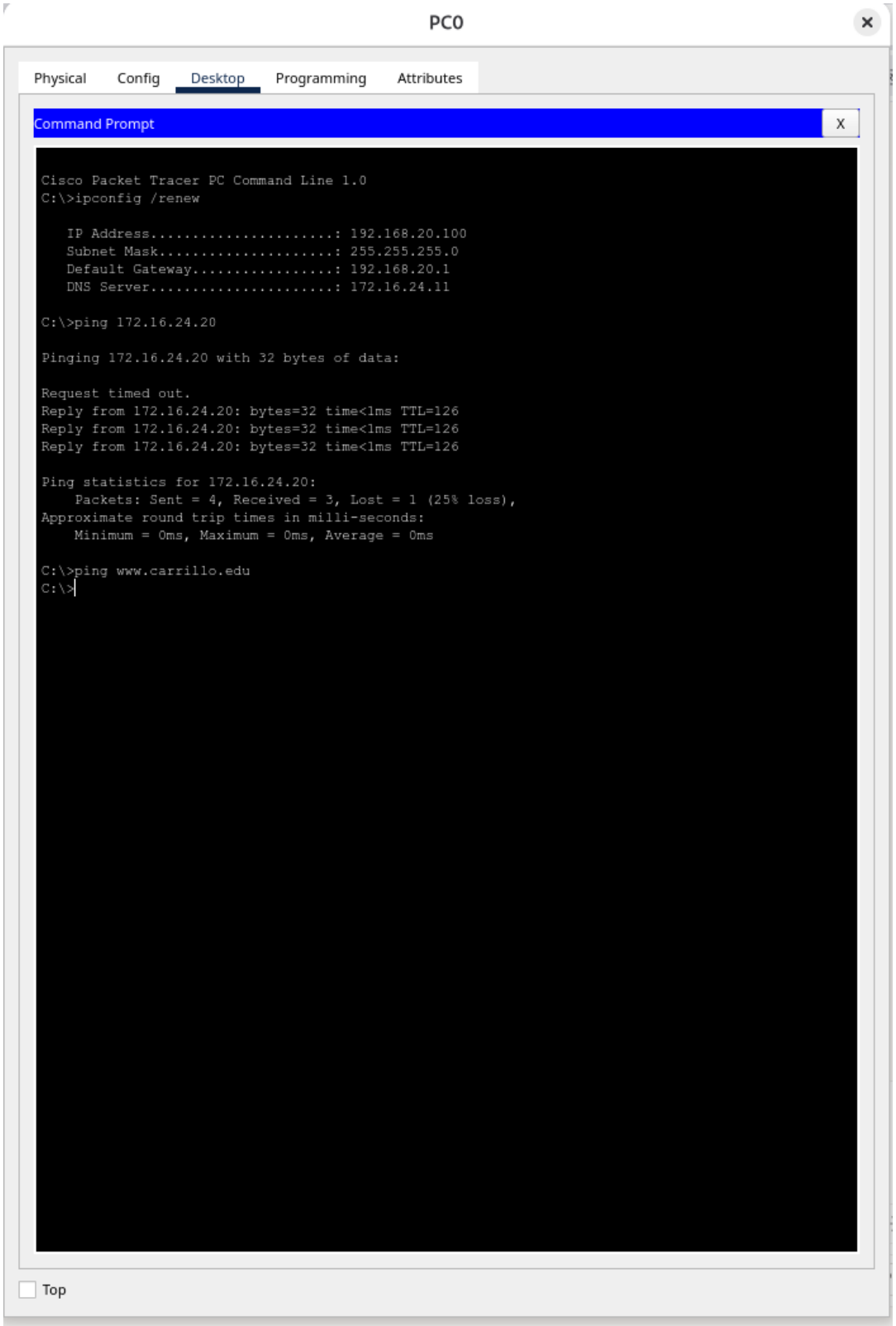
Il faut renseigner l'adresse du serveur DNS dans chacun des pools DHCP (et sauvegarder), puis renouveler le bail des postes.

Quand le champ DNS Server d'un pool n'est pas renseigné, le poste reçoit une adresse, un masque et une passerelle, mais **aucun serveur DNS** : il peut donc joindre un serveur **par son adresse IP**, mais pas **par son nom**.

On met donc **DNS Server =** `172.16.24.11` dans les trois pools (`serverPool`, `VLAN10`, `VLAN20`) et on clique `Save` — c'est ce que montre la capture des pools au §2.

Mesure faite sur PC0 (après `ipconfig /renew`) :

```
ping 172.16.24.20        ->  3/4 (1 perdu, 25 %) : l'adresse IP répond toujours
ping www.carrillo.edu    ->  aucune réponse : le nom n'est pas résolu
```



Option DNS retirée : le nom ne se résout plus

L'adresse du serveur DNS est diffusée **automatiquement à tous les postes** par le DHCP, sans rien configurer sur chacun d'eux.

---

7. Faire les tests d'accès aux serveurs web

Sur un poste, onglet **Desktop → Web Browser**, on tape l'URL complète (avec `http://`).

Test 1 — par adresse IP (le DNS n'intervient pas) :

`http://172.16.24.20` → la page du serveur s'affiche.

Test 2 — par nom (le DNS intervient) :

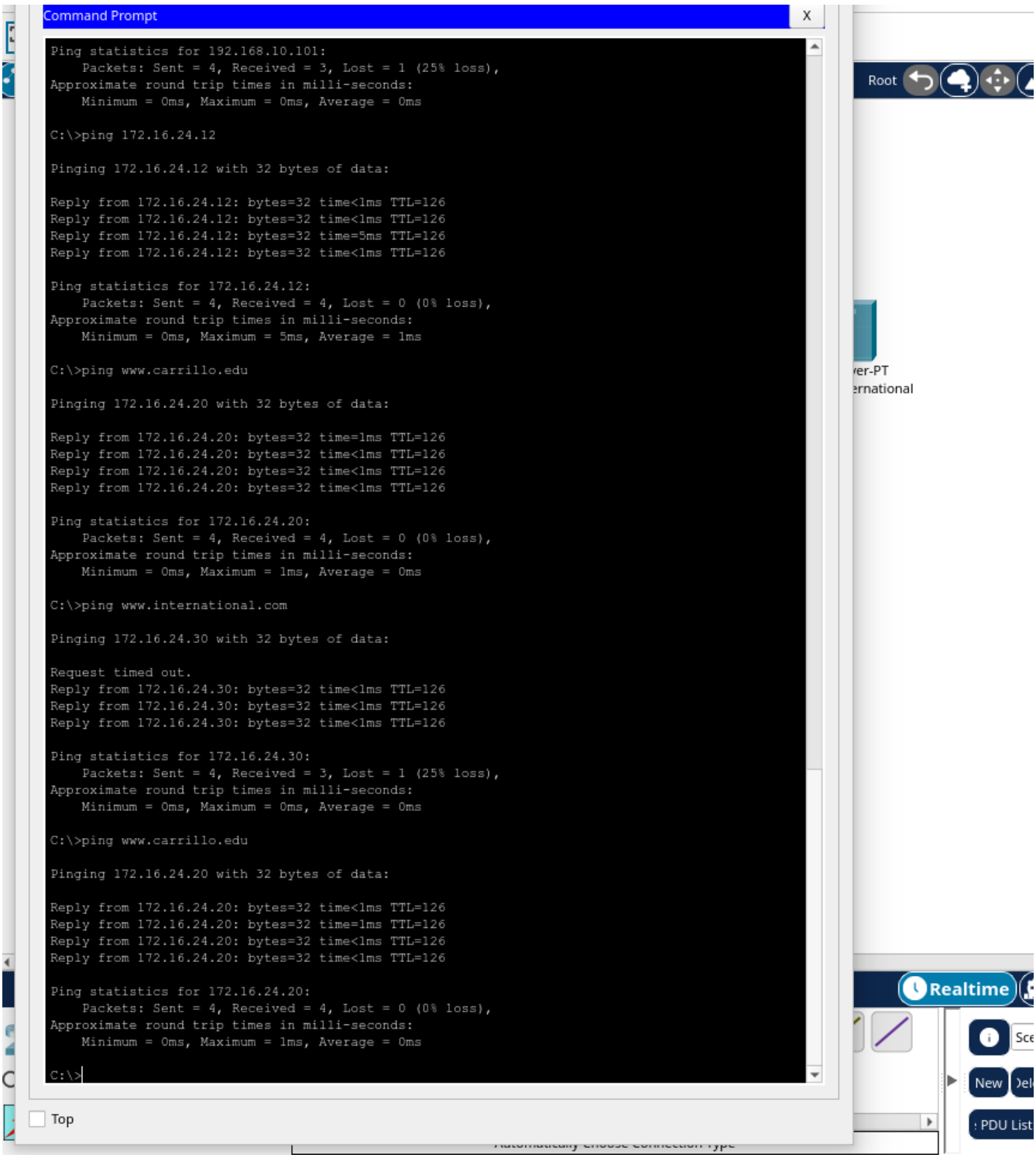
`http://www.carrillo.edu` → la même page :



PC0 : accès à www.carrillo.edu dans le navigateur

Test 3 — vérification de la résolution de noms depuis la Command Prompt :

```
ping www.international.com
ping www.carrillo.edu
```



PC0 : résolution des deux noms

Packet Tracer affiche avant le ping l'adresse trouvée :

Commande	Résolution	Résultat mesuré
`ping www.carrillo.edu`	`Pinging 172.16.24.20`	**4/4 (0 %)**, TTL 126, moy 0 ms
`ping www.international.com`	`Pinging 172.16.24.30`	3/4 (1 perdu, 25 % — 1er paquet ARP), TTL 126

Les deux noms sont donc bien résolus par le serveur DNS `172.16.24.11`, puis les échos ICMP partent vers les serveurs web (TTL 126 : les deux routeurs sont traversés).

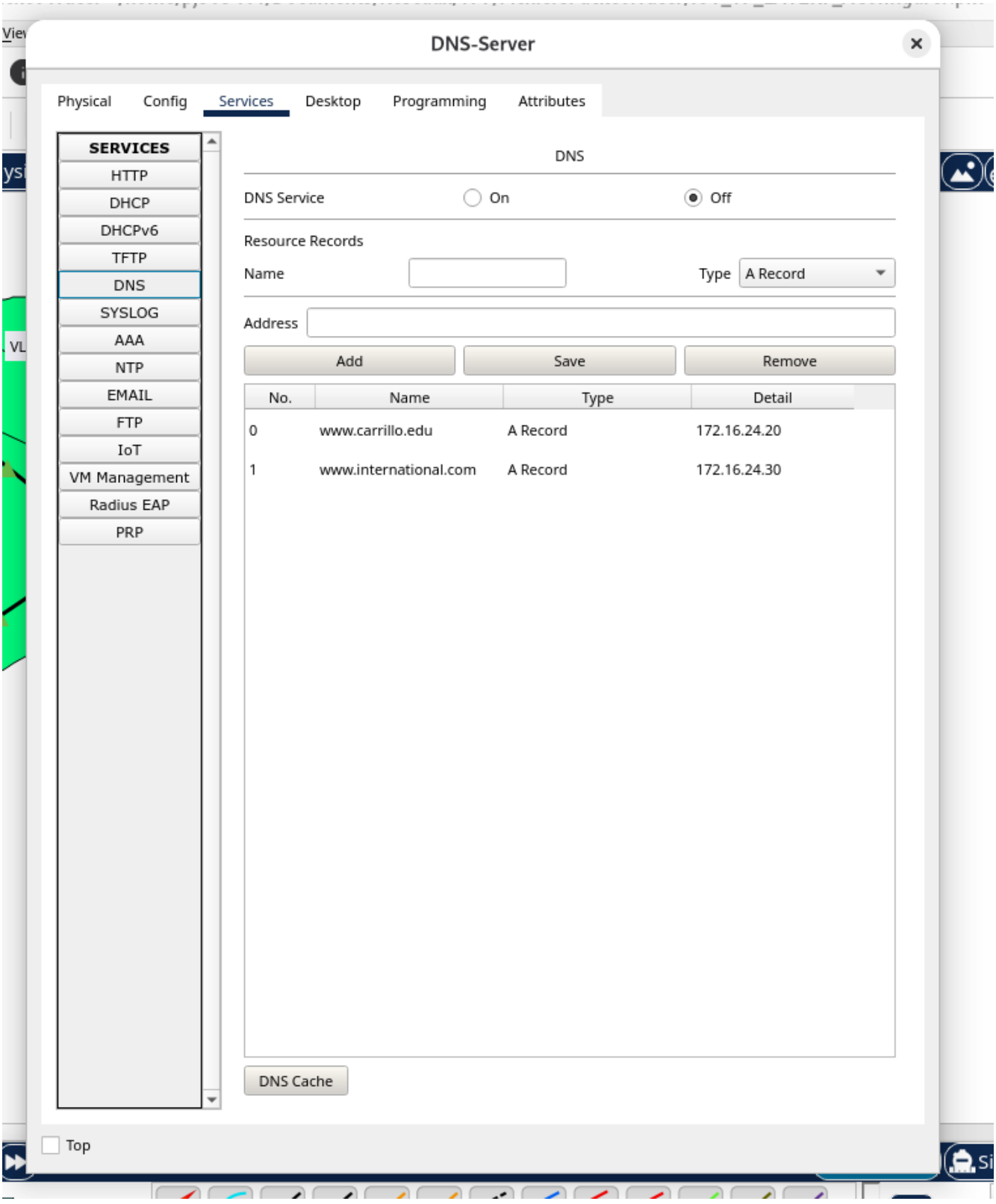
---

8. Séparez le serveur Web du serveur DNS. Qu'est-ce que cela change ?

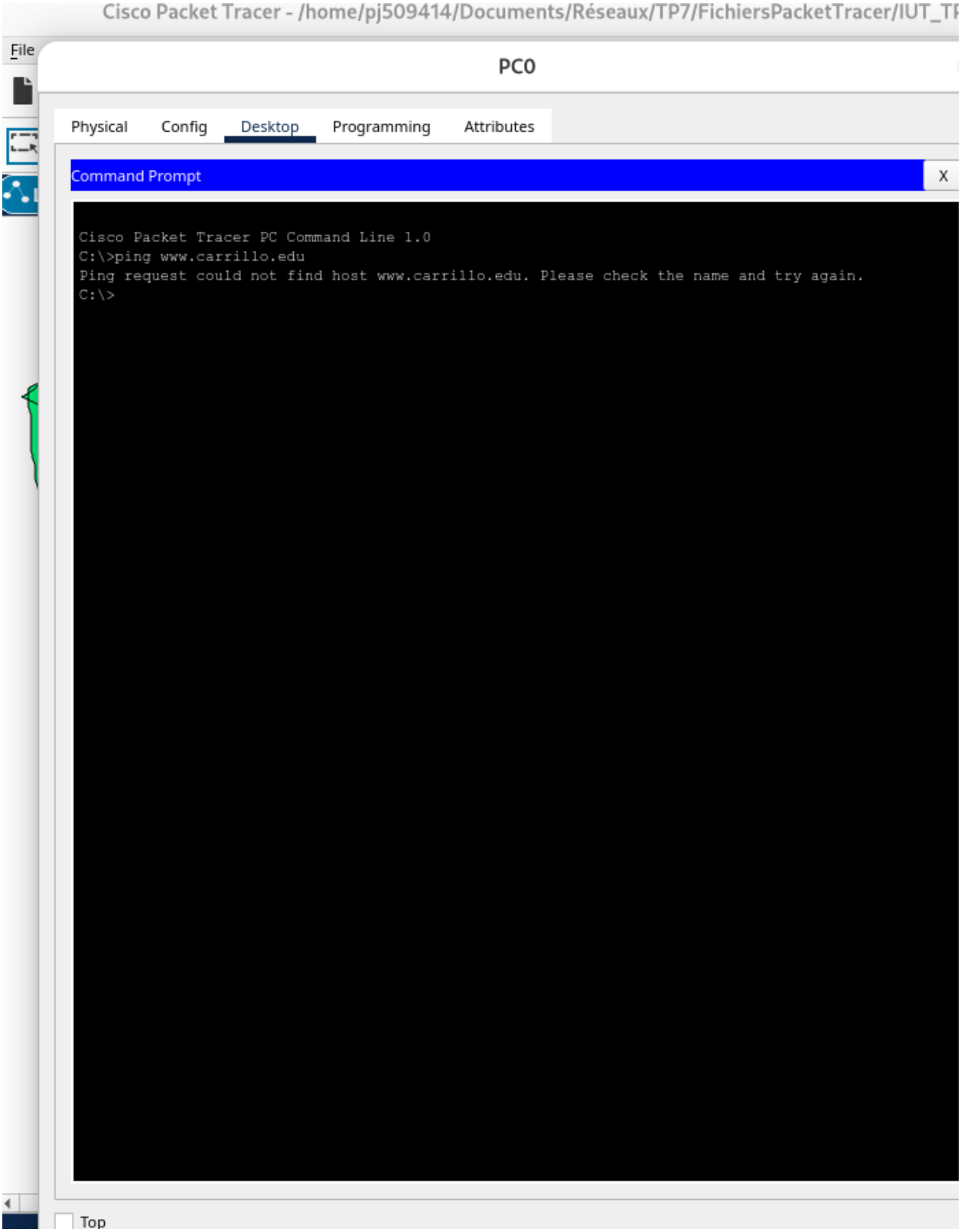
Réponse : cela fait deux machines distinctes — `172.16.24.11` **pour la résolution des noms,** `172.16.24.20` **et** `172.16.24.30` **pour les pages web.**

Dans la maquette, les rôles sont déjà séparés : le DNS est sur un serveur et les sites web sur deux autres. Ce que cela change : d'abord, **les services sont indépendants**, on l'a vérifié en éteignant le DNS, les noms ne se résolvent plus mais `http://172.16.24.20` continue de fonctionner, et inversement si un serveur web tombe le DNS continue de répondre, alors qu'avec un seul serveur pour tout la moindre panne coupe **tous** les services d'un coup. Ensuite, **la disponibilité et la charge** : le DNS reçoit beaucoup de requêtes très courtes tandis que le HTTP transfère de gros volumes, les séparer évite qu'ils se ralentissent mutuellement et permet de limiter ou de filtrer chacun séparément. **La configuration côté clients ne change pas** non plus : le navigateur et le système interrogent simplement le serveur DNS reçu par DHCP, que celui-ci soit sur la même machine que le serveur web ou non. **En contrepartie**, il faut administrer deux (ou trois) machines et réserver autant d'adresses IP fixes : c'est le prix de cette séparation.

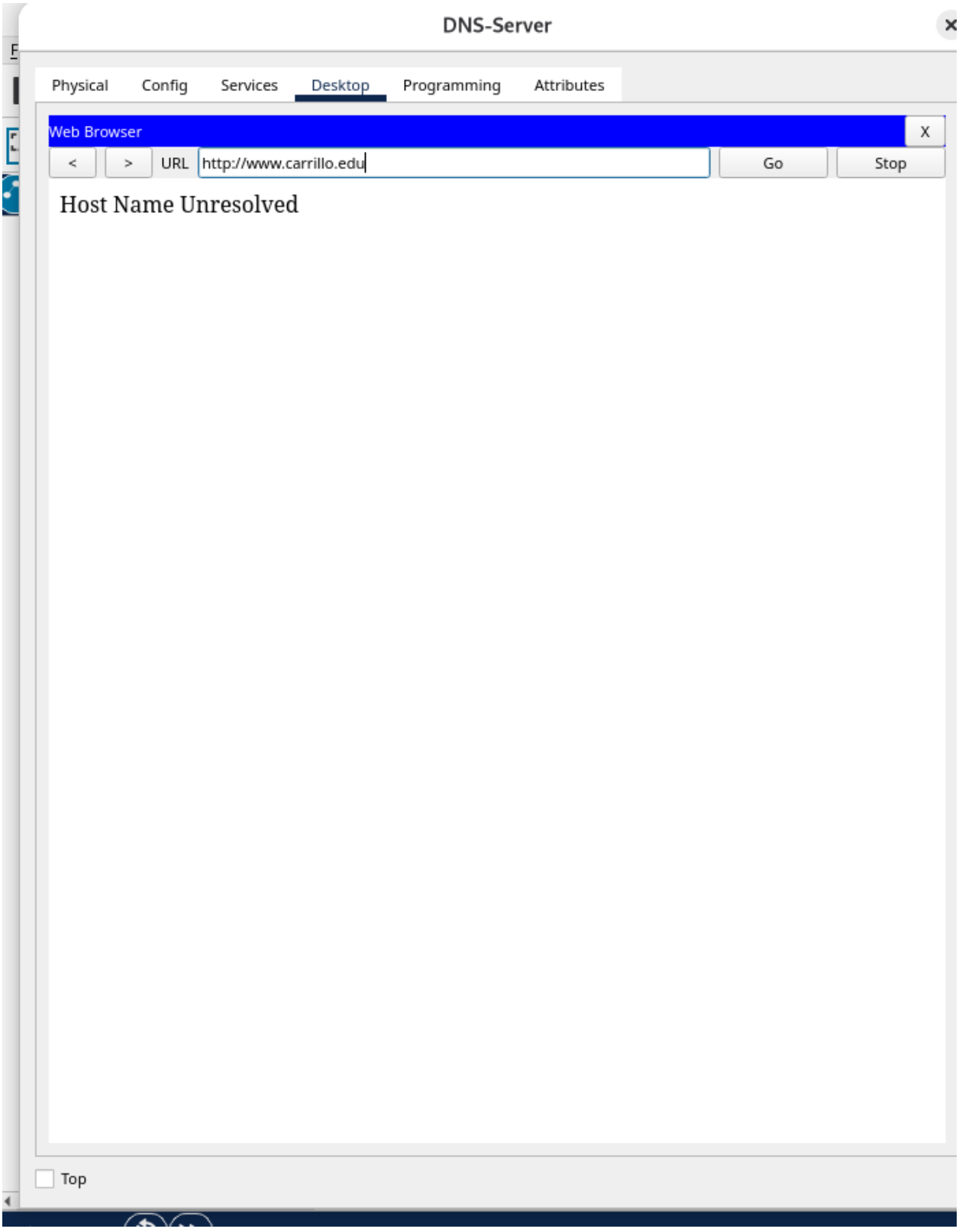
Vérification dans la maquette — avec `DNS Service : Off` sur `DNS-Server` (les deux enregistrements `A` restent dans la table) :



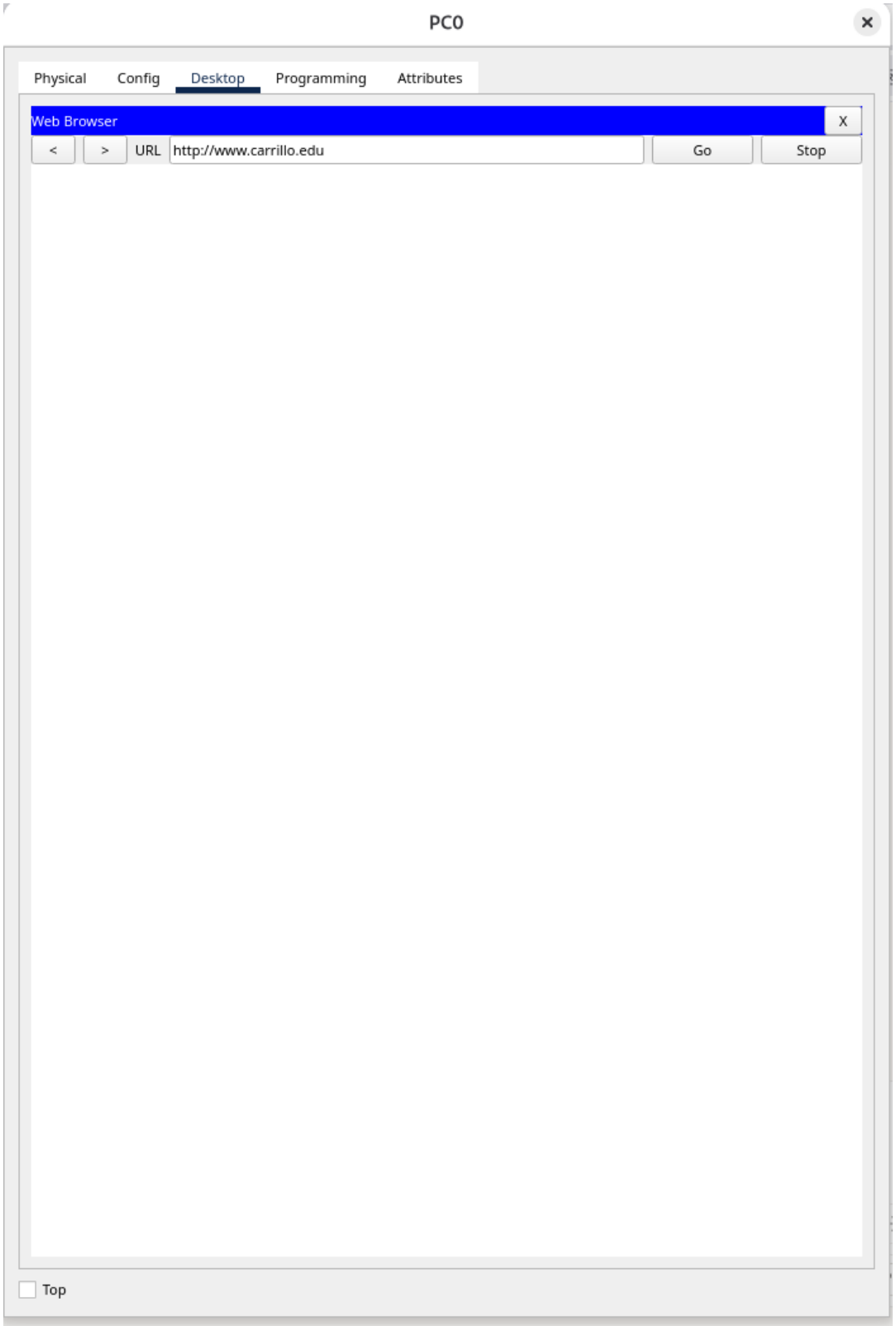
Service DNS éteint



Le nom échoue : ping



Le nom échoue : navigateur



Le nom échoue : navigateur, autre poste

Le ping d'un nom échoue avec le message exact `Ping request could not find host www.carrillo.edu. Please check the name and try again.` et le navigateur affiche `Host Name Unresolved.`

Alors que l'accès par adresse IP continue de fonctionner :



Le site par IP reste accessible sans DNS

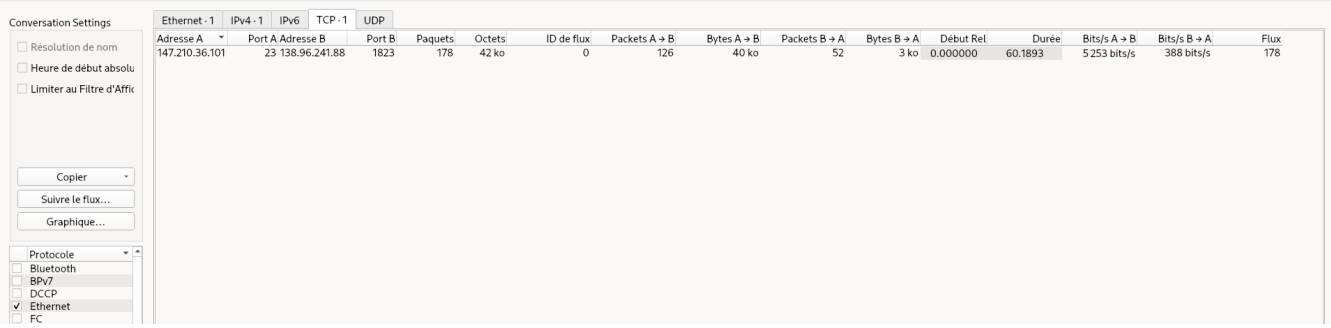
Partie II — Telnet

1. Quel est le protocole de transport utilisé pour cette connexion telnet ?

TCP, sur le port 23 (le port par défaut de Telnet).

2. Quelles sont les adresses IP du client et du serveur ?

Rôle	Adresse IP	Port
Client	`138.96.241.88`	1823 (port éphémère)
Serveur	`147.210.36.101`	23



Wireshark : la conversation TCP unique de la capture

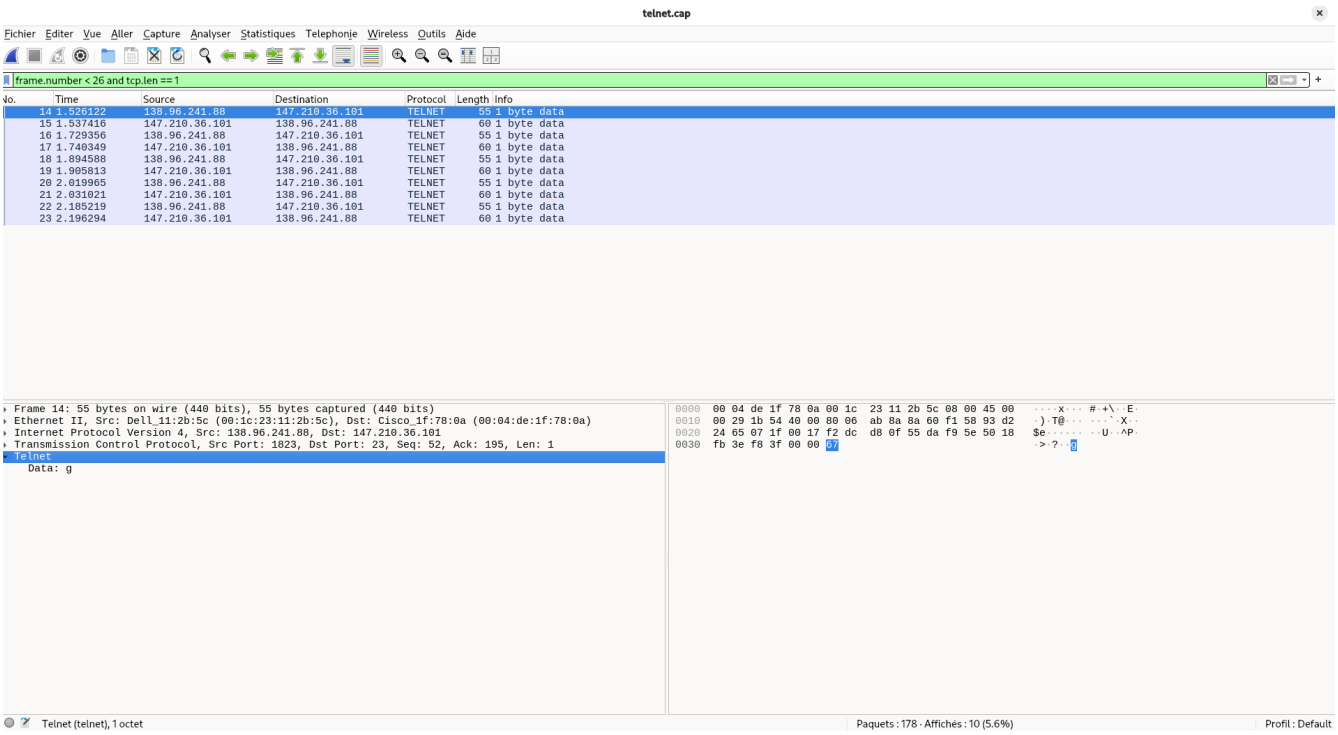
Une seule conversation TCP dans la capture : `138.96.241.88:1823` vers `147.210.36.101:23`, sur **178 paquets** et **60,1893 s**. Le découpage donné par Wireshark : **126 paquets / 40 ko** du serveur vers le client (les pages et l'écho des touches) contre **52 paquets / 3 ko** du client vers le serveur (les frappes et les acquittements TCP).

3. Quelles sont les informations qu'a consultées le client ? Filtre

```
ip.dst == 138.96.41.88
```

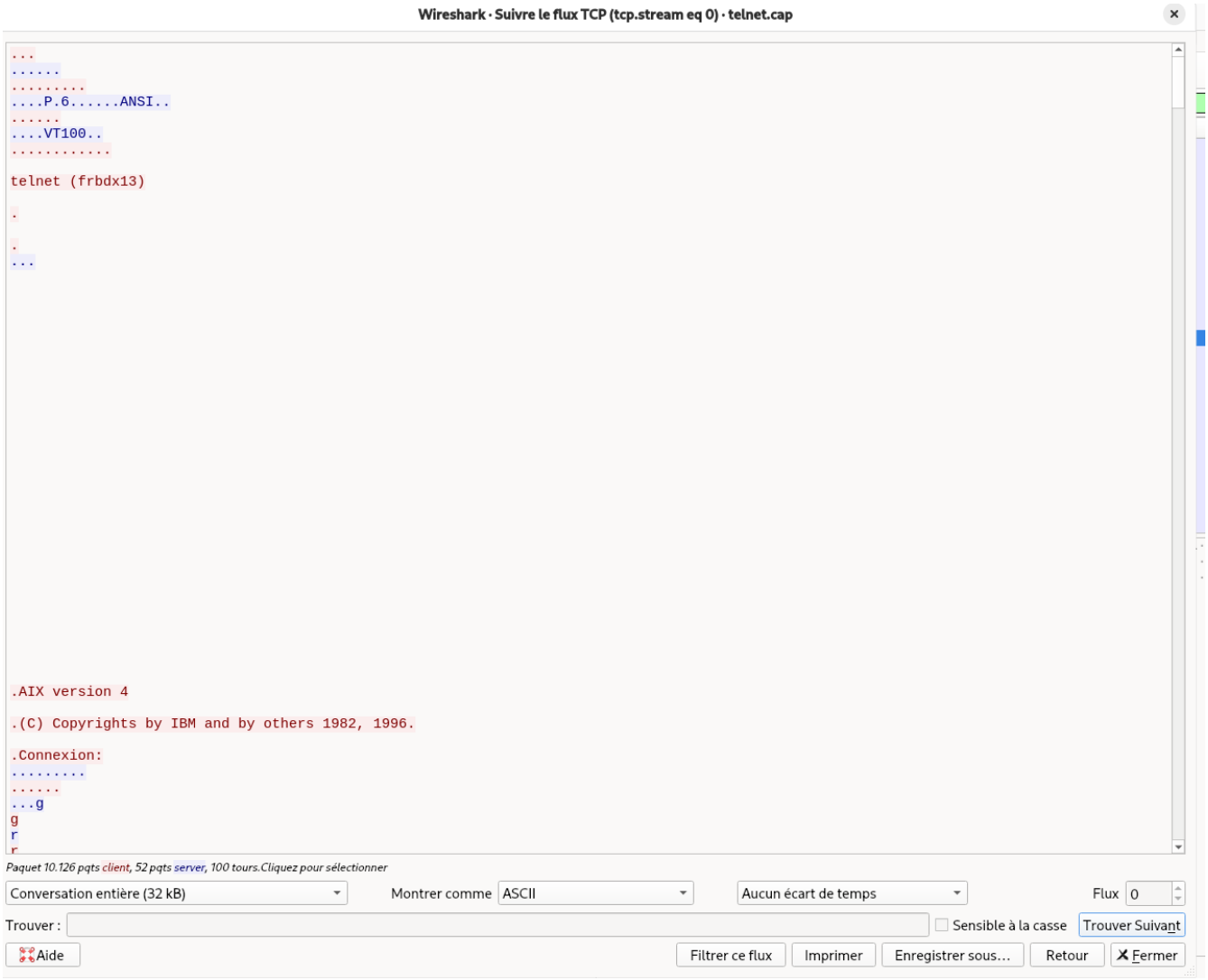
Le filtre `ip.dst == 138.96.241.88` garde tous les paquets **reçus par le client** : c'est exactement ce que le serveur lui a renvoyé, donc ce que le client a consulté. La session Telnet est transmise **en clair**, on peut donc lire tout son contenu.

Le login lui-même se lit caractère par caractère : chaque lettre de `grace` part dans un paquet d'un seul octet, que l'on peut ouvrir dans Wireshark.



Wireshark : la lettre « g » du login envoyée en clair

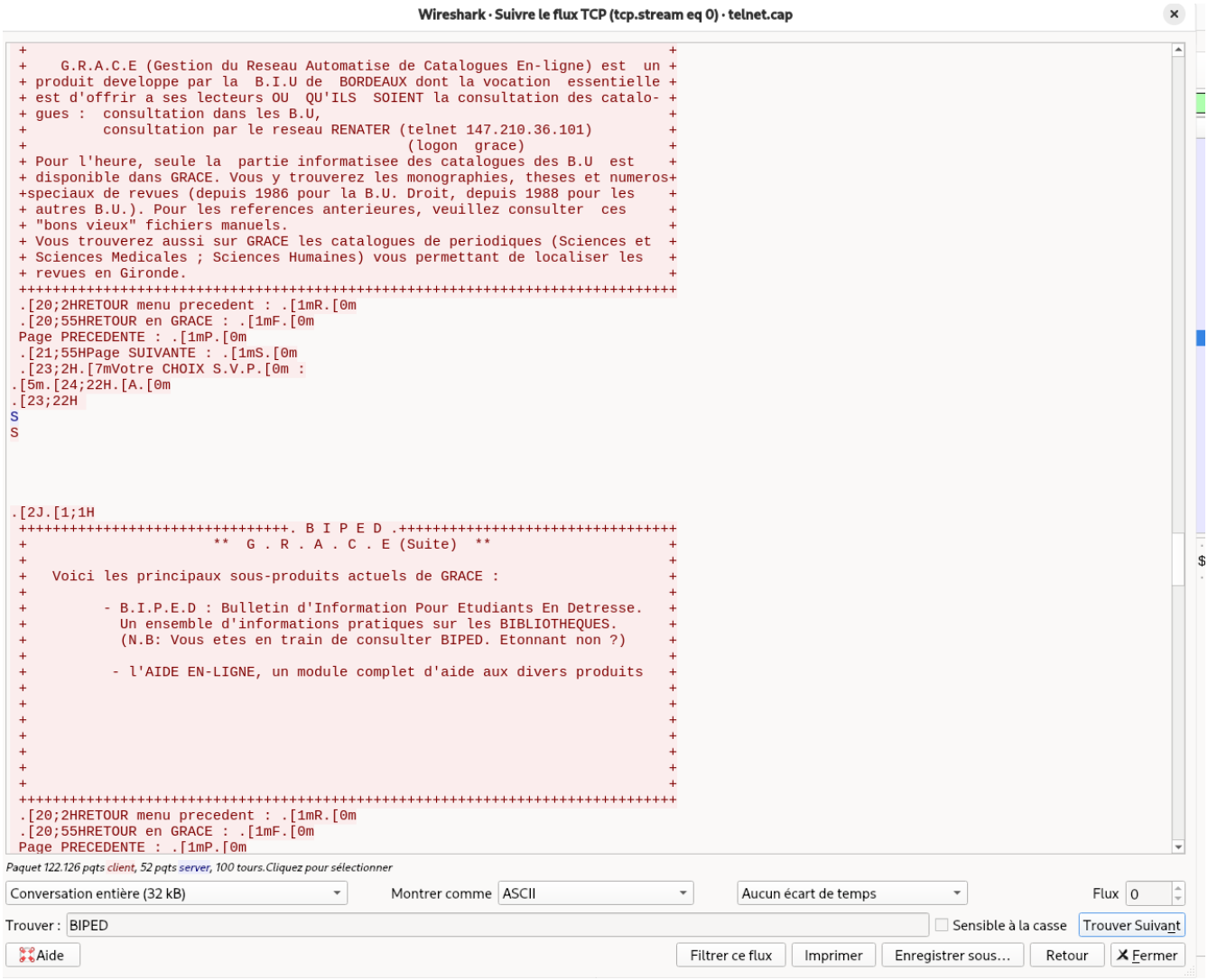
En suivant le flux TCP de la session, Wireshark reconstitue tout le texte échangé, sans rien déchiffrer : le début de la session est directement lisible.



Wireshark : la session reconstituée en texte clair

Les informations consultées, dans l'ordre : d'abord l'en-tête de la ligne Telnet `telnet (frbdx13)` puis `AIX version 4` / `(C) Copyrights by IBM and by others 1982, 1996.`, donc le serveur est une machine **IBM AIX 4** ; ensuite `Connexion: grace`, qui donne le **nom d'utilisateur** `grace`, suivi de la `Dernière connexion: jeu 21 fév 17:01:53 NFT 2008 le /dev/pts/0 à partir de bougeotte.inria.fr` (date et poste de la connexion précédente) ; puis la bannière de l'application **GRACE** (Gestion du Réseau Automatisé de Catalogues En-ligne), version 3.60, le `21/02/2008 - 17:04`, avec son menu (AIDE, JOURNAL, INFORMATION, QUITTER) ; puis l'**aide en ligne** en 3 pages (`GENERALITES 1/3`, `2/3`, `AIDE GENERALE 3/3`), parce que le client a tapé `A` ; puis le **B.I.P.E.D.** (Bulletin d'Information pour Étudiants en Détresse) et ses rubriques (`L'INFORMATIQUE à la B.U.`, `L'AVENIR INFORMATIQUE`, `G.R.A.C.E`, `G.R.A.C.E (suite)`, `SIBIL`, `Les MICRO-ORDINATEURS pour la R.D.I.`) ; et enfin le **JOURNAL** (« DERNIERES NOUVELLES de GRACE ») puis la page de sortie « GRACE vous remercie de votre visite » avec l'horloge du serveur `17:05:41`.

Plus bas dans le même flux, on retrouve la page de présentation de GRACE et le début du bulletin **B.I.P.E.D.** (`G.R.A.C.E (Suite)`, SIBIL, l'aide en ligne) :



Wireshark : les pages consultées dans la session

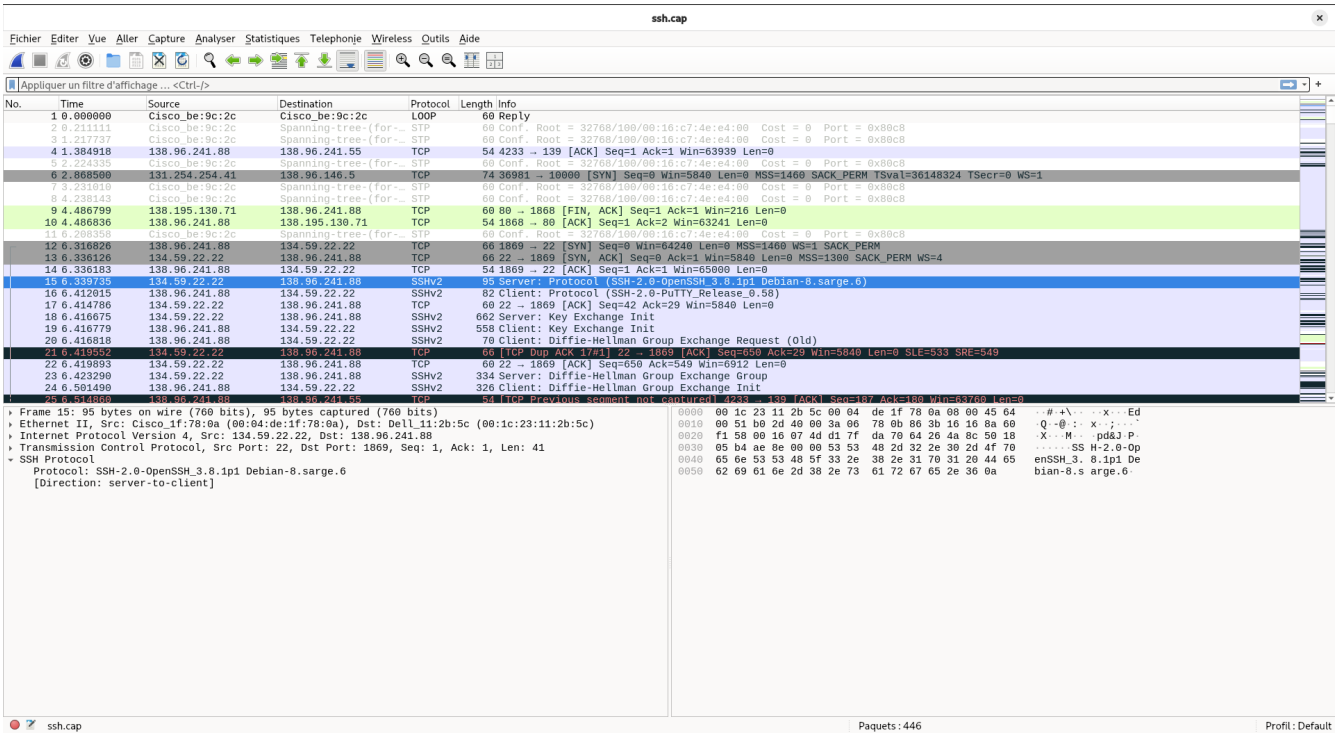
4. Expliquez le résultat obtenu

Ce sont des **informations textuelles** du catalogue documentaire qui circulent en clair : rien n'est chiffré. Sur 178 paquets, **126 vont vers le client** — le serveur renvoie tout ce qu'il affiche, plus l'**écho de chaque caractère** tapé (les paquets d'un seul octet `g`, `r`, `a`, `c`… quand l'utilisateur saisit `grace`) et les acquittements TCP, parce que c'est le **terminal** de l'utilisateur qui est renvoyé à distance, écran par écran. Le client, lui, n'a envoyé que **117 octets** au total : son login, ses quelques touches (`A`, `S`, `R`, `I`, `J`, `F`, `Q`) et les retours de ligne. Les pages contiennent aussi des **codes de commande du terminal** (couleurs, positionnement du curseur, effacement d'écran) : en les retirant, on lit exactement ce que l'utilisateur avait à l'écran. Telnet transmet donc **tout en clair**, jusqu'au mot de passe : une simple capture réseau suffit à reconstituer une session et à récupérer des identifiants, c'est la raison pour laquelle Telnet est remplacé par **SSH**.

Partie III — SSH

1. À quel serveur s'est connecté le client ?

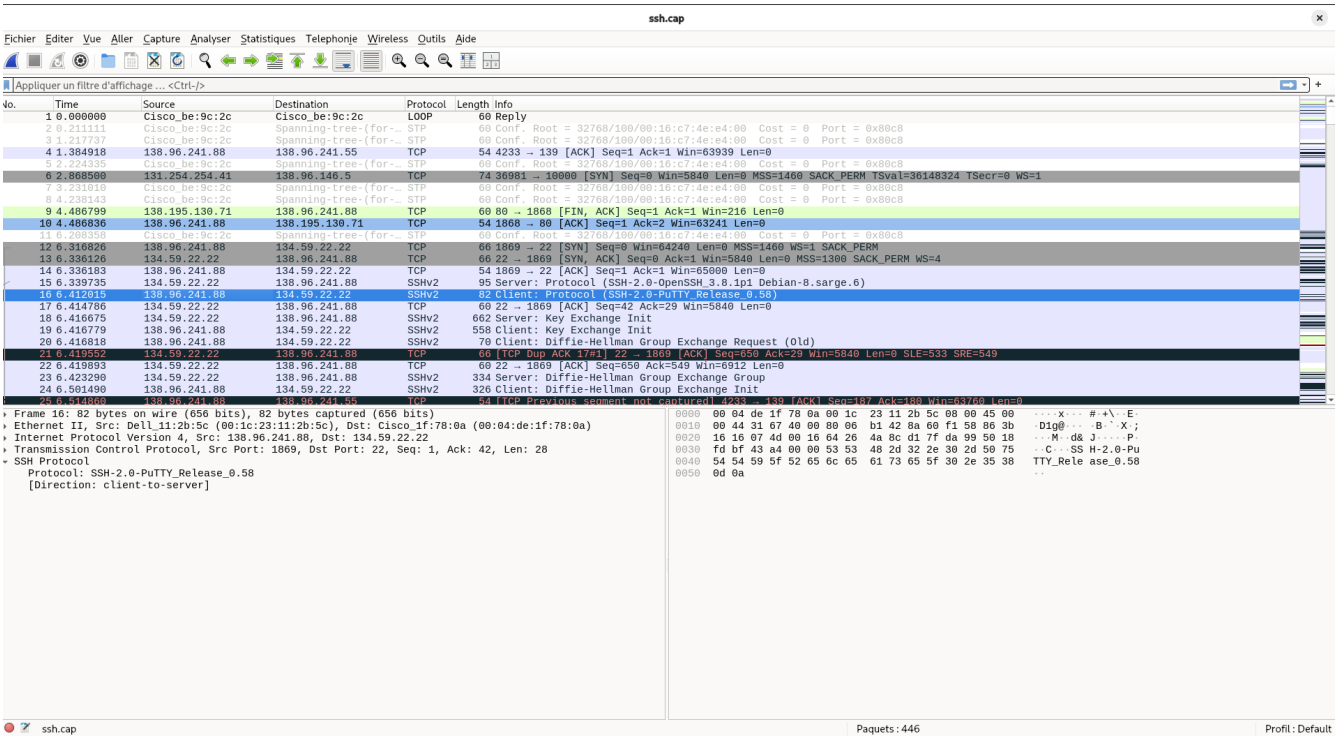
Au serveur `134.59.22.22`**, port 22**, d'après la première chaîne envoyée par le serveur : `SSH-2.0-OpenSSH_3.8.1p1 Debian-8.sarge.6`. Le client, lui, utilise le port éphémère `1869`. La session représente **319 paquets** et **144 537 octets** — beaucoup plus que la session Telnet, qui ne transportait que du texte.



Wireshark : le serveur annonce OpenSSH 3.8.1p1

2. Le client a-t-il utilisé un logiciel ? Si oui, lequel ?

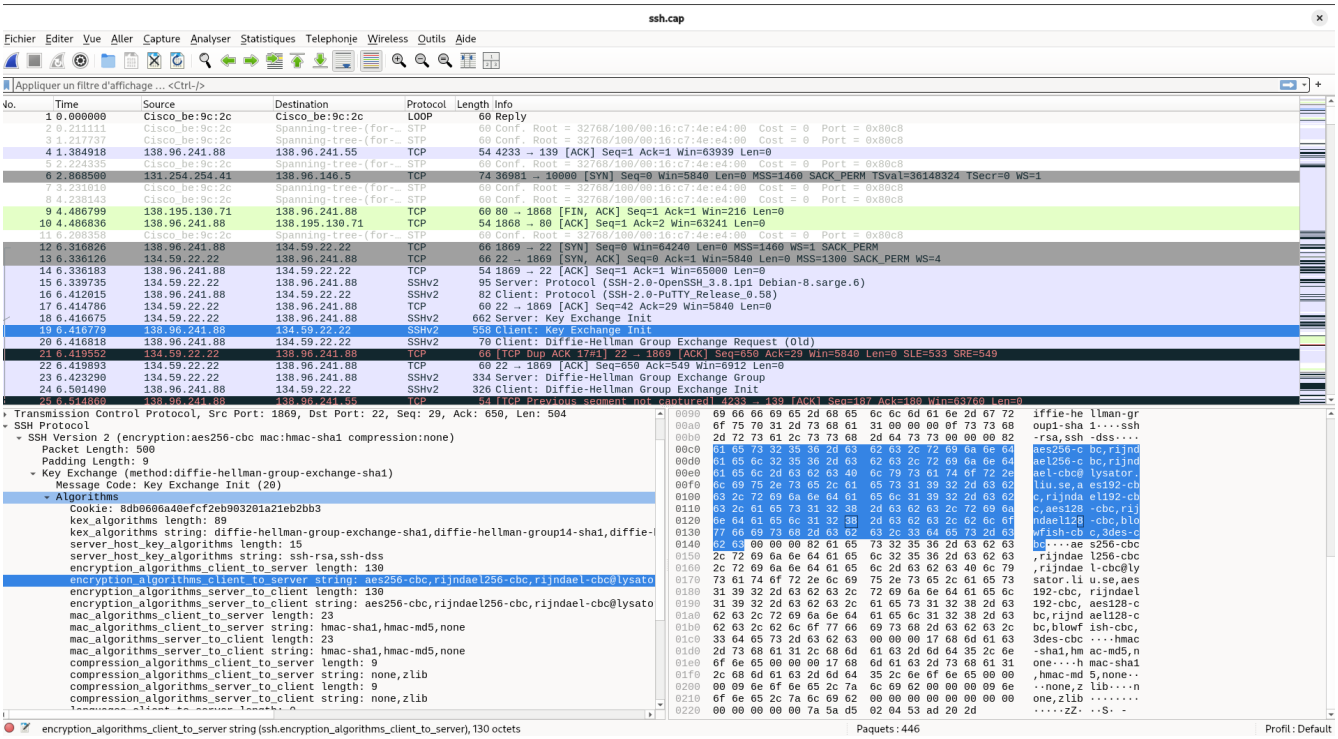
Oui : PuTTY release 0.58. La chaîne d'identification du client est `SSH-2.0-PuTTY_Release_0.58`, affichée en clair au début de la session.



Wireshark : le client s'annonce PuTTY 0.58

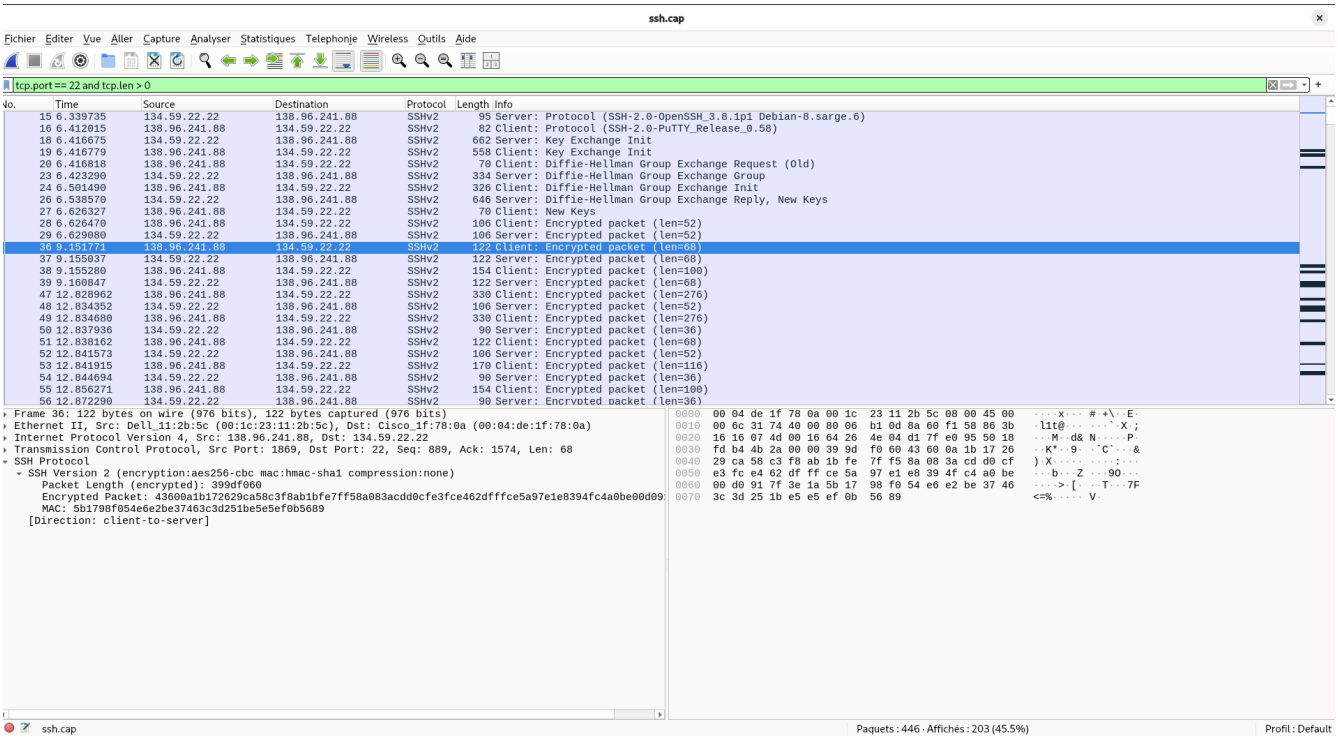
3. Quelles sont les informations qui circulent ? Commenter.

a) Avant le chiffrement, tout est lisible : les deux programmes s'annoncent (`SSH-2.0-PuTTY_Release_0.58` côté client, `SSH-2.0-OpenSSH_3.8.1p1 Debian-8.sarge.6` côté serveur) et **négocient les algorithmes**, à savoir `diffie-hellman-group-exchange-sha1`, `diffie-hellman-group14-sha1` et `diffie-hellman-group1-sha1` pour l'échange de clés, `ssh-rsa` et `ssh-dss` pour les clés d'hôte, `aes256-cbc`, `aes192-cbc`, `aes128-cbc`, `3des-cbc`, `blowfish-cbc` et `arcfour`... pour le chiffrement, `hmac-sha1` et `hmac-md5` pour l'intégrité, `none` et `zlib` pour la compression.



Wireshark : la négociation des algorithmes, en clair

b) Après l'échange de clés, plus rien n'est lisible : l'authentification de l'utilisateur, le mot de passe et les commandes sont **chiffrés**. Les algorithmes retenus sont ceux négociés au début : `aes256-cbc` pour le chiffrement et `hmac-sha1` pour l'intégrité.



Wireshark : les paquets suivants sont chiffrés

C'est toute la différence avec Telnet : SSH **chiffre** la session (ici AES ou 3DES) et **vérifie son intégrité** (HMAC), donc une capture ne permet plus de lire ce qui circule — seul l'entête de négociation reste visible, le reste apparaît comme des octets pseudo-aléatoires.

	Telnet	**SSH**
Port	23	22
Session	**en clair**	**chiffrée** après la négociation
Ce qu'on lit dans une capture	login, mot de passe, menus, données	seulement les bannières et la liste des algorithmes

C'est pourquoi on utilise SSH plutôt que Telnet (et `sftp`/`scp` plutôt que `ftp`) : la confidentialité et l'intégrité de la session sont assurées.